



Entwicklung eines quadrupeden Roboters mit aktiver Wirbelsäule und quasi-direktem Antrieb

Development of a Quasi-Direct Drive Spinal Quadruped Robot

Wissenschaftliche Arbeit zum Erlangen des akademischen Grades

B. Sc.

an der School of Engineering and Design der Technischen Universität München

Themenstellender: Prof. Dr.-Ing. Johannes Fottner
Lehrstuhl für Fördertechnik Materialfluss Logistik

Betreuer/Betreuerin: Mr. Lingchong Gao and Mr. Liangyu Dong

Eingereicht von: Federico Cominelli (, B. Sc.)
Boltzmannstrasse 15
85748 Garching bei München
+33 6 43 91 76 13

Eingereicht am: 01.09.2025 in Garching

Inventarnr. fml: 2025/78

Vorwort

Die vorliegende Arbeit entstand unter der wissenschaftlichen und inhaltlichen Anleitung von Mr. Lingchong Gao, wissenschaftlicher Mitarbeiter am Lehrstuhl für Fördertechnik, Materialfluss und Logistik (fml) der Technischen Universität München, sowie von Mr. Liangyu Dong am Lehrstuhl für Robotik, Künstliche Intelligenz und Echtzeitsysteme der Technischen Universität München.

Vereinbarung zum Nutzungsrecht

Hiermit gestatte ich dem Lehrstuhl für Fördertechnik Materialfluss Logistik diese Studienarbeit bzw. Teile davon nach eigenem Ermessen an Dritte weiterzugeben, zu veröffentlichen oder anderweitig zu nutzen. Mein persönliches Urheberrecht ist über diese Regelung hinaus nicht beeinträchtigt. Eventuelle Geheimhaltungsvereinbarungen über den Inhalt der Arbeit zwischen mir bzw. dem Lehrstuhl für Fördertechnik Materialfluss Logistik und Dritten bleiben von dieser Vereinbarung unberührt.



Garching, 01.09.2025

Abstract

This thesis presents the development of a quasi-direct drive quadruped robot with a three-degree-of-freedom spine, designed to serve as a research platform for future studies in locomotion and control. The quadruped was engineered to be mechanically robust enough to perform stable walking and balancing while integrating additional flexibility through its articulated spine. The work involved mechanical design and manufacturing, the integration of high-performance actuators and sensors, and the development of embedded control software using one Raspberry Pi 5 and the ROS 2 framework. A MuJoCo-based simulation environment was created to validate control strategies before hardware testing. The robot successfully demonstrated two key capabilities: trotting at a constant speed of 4 cm/s using a hard-coded closed-loop trajectory, and balancing on an inclined plane. The software's modular design allows for easy implementation of advanced gait generation, real-time optimization, and spine-assisted locomotion strategies.

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Related Work	1
1.3	Goal of this Thesis	3
1.4	Structure of the Thesis	4
2	Hardware Design	6
2.1	General Overview	6
2.2	Actuator Choice	8
2.2.1	Actuator Choice	8
2.3	Electronics Hardware	9
2.3.1	Electronics choice	9
2.3.2	Electronics Architecture	11
2.3.3	Communication frequency	12
2.3.4	USB communication protocol	12
2.3.5	FDCAN communication protocol	13
2.3.6	Damiao motor commands	15
2.4	Mechanical Design	16
2.4.1	Frame and supports	16
2.4.2	Leg Design	17
2.4.3	Spine Design	21
3	Control	26
3.1	Approach	26
3.2	Inverse Kinematics and Forward Kinematics of the leg	26
3.2.1	Forward Kinematics	28
3.2.2	Inverse Kinematics	29
3.3	Inverse Kinematics and Forward Kinematics of the spine	32
3.3.1	Inverse Kinematics	32
3.3.2	Forward Kinematics	35

3.4	Gait: trot equations	36
3.5	Balance on an inclined surface	37
3.5.1	Rear body balance	38
3.5.2	Front body balance	38
3.5.3	Legs	39
4	Software Design	41
4.1	Introduction	41
4.2	Joints Notation	41
4.3	ROS 2 architecture	42
4.3.1	leg_command_node.cpp	42
4.3.2	mujoco_node.cpp	44
4.3.3	hardware_check.cpp	45
4.3.4	trot_node.cpp	47
4.3.5	spine_pitch_inclined_plane_node.cpp	49
4.4	MuJoCo Simulation	52
4.4.1	Control	53
4.4.2	XML Models	53
4.4.3	Sensor IDs	53
4.4.4	Inclined Plane	54
5	Testing and Results	55
5.1	Testing Procedure and Recommendations	55
5.1.1	MIT Position Limits	55
5.1.2	Communication Setup	55
5.1.3	Useful Commands	55
5.1.4	Final Recommendations	56
5.2	Results	56
5.2.1	Hardware Analysis	56
5.2.2	Test cases choice	59
6	Conclusion	62

Literaturverzeichnis	63
7 List of Figures	66
8 List of Tables	68
A Anhang	A-1
A.1 Code Listings	A-1

1 Introduction

1.1 Introduction

Quadruped robots have gained significant attention in recent years due to their versatility, mobility, and potential applications in environments that are inaccessible or hazardous to humans. Their ability to traverse uneven terrain and maintain stability under perturbations makes them valuable platforms for research in locomotion control, and robotic autonomy.

This thesis focuses on the development of a quasi-direct drive quadruped robot with a three-degree-of-freedom spine, designed as a research platform for future work on locomotion and control strategies.

1.2 Related Work

The development of quadruped robots has progressed rapidly over the past two decades. Early prototypes in the late 1990s and early 2000s, such as Raibert's quadrupeds at the MIT Leg Lab, were primarily research platforms used to explore basic locomotion and gait control strategies [Rai-1986], [Lee-1996]. At that time, most quadrupeds relied on relatively simple gait generators and rigid torsos, but they already demonstrated the potential of legged systems to traverse uneven terrain where wheeled robots fail. A major milestone was reached with Boston Dynamics BigDog (2005 - 2008), which introduced hydraulically actuated legs and showed unprecedented robustness on rough terrain, pushing quadrupeds from laboratory prototypes toward field-capable machines [Rai-2008]. During the 2010s, progress accelerated with the introduction of more advanced actuators and control frameworks, leading to robots such as HyQ (2011) [Sem-2011] and ANYmal (2016) [Hut-2016]. Since then, quadrupeds have become increasingly refined, with better actuators, more powerful control algorithms, and improved mechanical designs, enabling robust dynamic locomotion and even commercial applications.

Among the most famous quadrupeds today are ANYmal from ETH Zurich, the MIT Cheetah family, and HyQ from the Italian Institute of Technology. ANYmal is torque-controlled and uses serieselastic actuation combined with wholebody control, which allows it to perform robust and dynamic movements [Hut-2016], [Bel-2017]. HyQ is hydraulically actuated and showed that high force and robustness could be achieved in quadruped robots [Sem-2011]. The MIT Cheetah series pushed the field further by introducing proprioceptive electrical actuators and advanced model predictive control (MPC), allowing dynamic running and jumping behaviors [Ble-2018], [DiC-2018]. Smaller and more accessible platforms were later introduced, such as the MIT Mini Cheetah [Kat-2019] and Stanford Doggo [Kau-2019], which used quasidirect drive actuators to lower cost and complexity while still enabling dynamic gaits. Reinforcement learning has also started to play an important role, for example with ANYmal, where learning-based controllers demonstrated agile locomotion directly on

hardware [Hwa-2019]. Finally, Boston Dynamics Spot is an example of how quadrupeds have moved from research into commercial use, with reliable autonomy for inspection and mapping tasks.

Table 1-1 summarizes the specifications of some of these quadrupeds and compares them with the ASTRA prototype developed in this thesis.

Table 1-1: Specifications of selected quadrupeds compared to the developed prototype.

Quadruped	Mass [kg]	Dim (l,w,h) [mm]	Max Speed [m/s]	Autonomy [min]
MIT Mini Cheetah	9.0	$480 \times 270 \times 300$	3.9	30–120
ASTRA	9.5	$565 \times 287 \times 435$	0.04	25
ANYmal	50	$930 \times 530 \times 890$	1.3	90
Spot	32.7	$1100 \times 500 \times 700$	1.6	90



Figure 1-1: From left to right: MIT Mini Cheetah (MIT), ANYmal (ETH Zurich), Spot (Boston Dynamics).

As rigid-torso quadrupeds have reached a very advanced stage of development, with some even commercialized, research interest has started to shift towards robots with active spines. In nature, animals such as cheetahs use their flexible spine to increase stride length, running speed, and efficiency. This biological inspiration motivates the design of quadrupeds with actuated spines.



Figure 1-2: Spine flexion and extension in a running cheetah.

Initial studies explored very simple spine designs. For example, early prototypes with only one degree of freedom (DoF) in the sagittal plane showed that an actuated spine can support bounding locomotion [Lee-1996]. Later experiments confirmed that bending the spine could increase speed and improve stability, although sometimes at the cost of efficiency [Kho-2013]. Other works, such as the Inu and Canid robots from Koditschek Lab, introduced a parallel elastic and powered spine, demonstrating experimentally that spinal actuation helps with self-manipulation and with overcoming obstacles [Dup-2017], [Pus-2013]. At the same time, several groups studied active spines in simulation, using trajectory optimization or simplified models to show potential benefits of twisting, extending, or prismatic spines [Ye-2021], [Cap-2023].

More recently, researchers have built quadrupeds with spines of multiple DoFs. The Stoch 2 robot included a two-DoF spine and applied reinforcement learning to learn bounding behaviors that actively used the spine [Bha-2019]. Yat-sen Lion incorporated a three-DoF spine, where the control problem was addressed by extending model predictive control to handle the changing center of mass and inertia of the body [Li-2023]. STRAY represents one of the most advanced prototypes: it uses a four-DoF symmetric five-bar spine mechanism and a hybrid control approach combining trajectory references with reinforcement learning [Wan-2024]. These studies show that while spine actuation can sometimes increase energy cost at low speeds, it tends to provide clear advantages for more dynamic tasks, such as high-speed running, long jumps, and wall-assisted leaps [Dup-2017], [Cap-2023], [Wan-2024].

In this context, the present work introduces ASTRA, a lightweight research quadruped designed with a modular spine that can behave either as a rigid torso or as an actuated joint. This allows direct comparison between rigid and active torso designs on the same platform. Furthermore, the electronics and software were built to support both model-based control strategies (such as MPC [DiC-2018], [Li-2023]) and learning-based methods (such as RL [Hwa-2019], [Wan-2024]), providing a flexible testbed for future research on how spines should be controlled in quadrupeds.

1.3 Goal of this Thesis

The primary objective of this thesis is to design and build a spinal quadruped robot ensuring that it is robust enough to walk, balance, and serve as a reliable foundation for further research.

To achieve these objectives, the work covers:

- The manufacturing and design of the mechanical structure and electronic components.
- Integration of quasi-direct drive actuators and an IMU.
- Development of control software using a Raspberry PI 5 and ROS 2.
- Simulation-based validation using MuJoCo before hardware testing.
- Experimental verification through trotting and balancing tasks.

1.4 Structure of the Thesis

The thesis is organized in a chronological manner, beginning with the design and assembly phase, followed by the derivation of kinematics and control equations, then the description of the software architecture, and finally the presentation of experimental results:

- **Chapter 1** introduction
- **Chapter 2** details the mechanical and electronics design.
- **Chapter 3** presents the kinematics derivations, and control strategies for the two test cases (trot and balance).
- **Chapter 4** describes the software architecture, ROS 2 nodes, and the MuJoCo simulation setup.
- **Chapter 5** outlines the testing procedures, and experimental results.

The quadruped presented is the following:

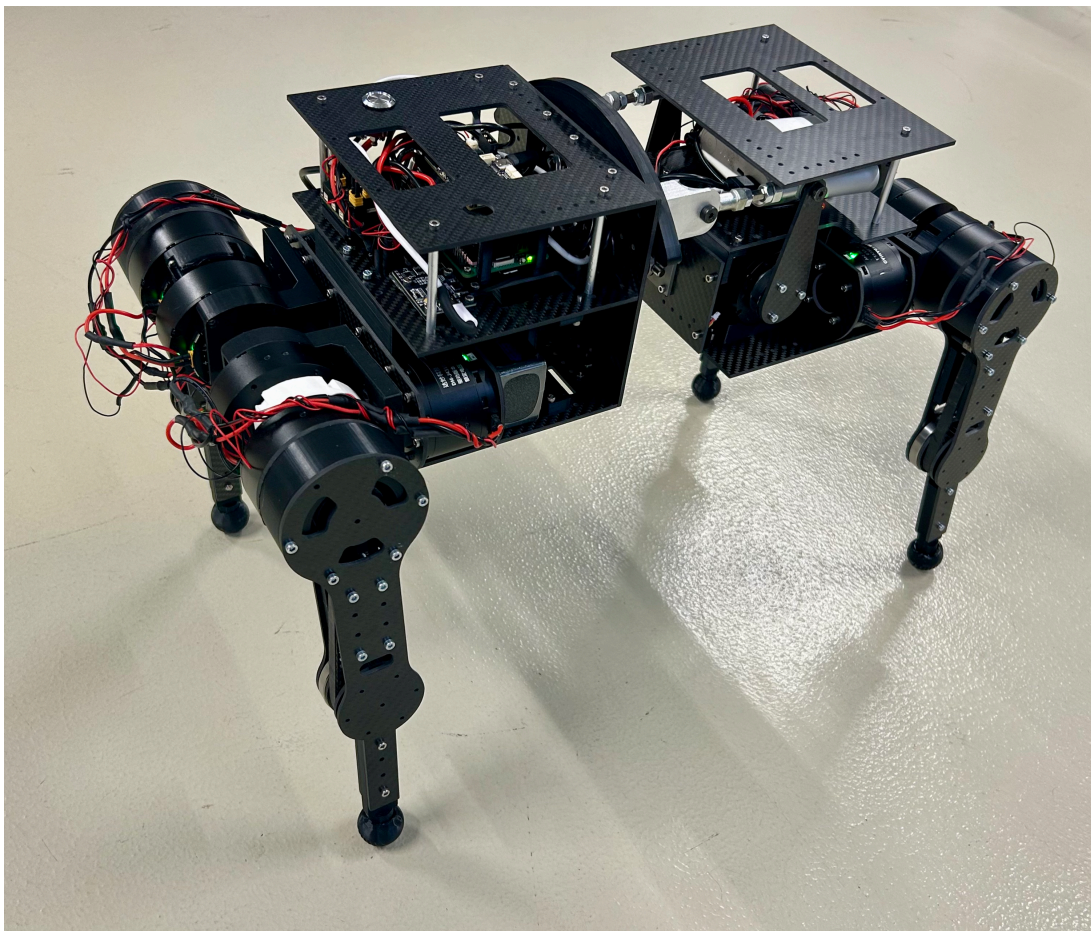


Figure 1-3: Picture of the quadruped without sideplates

It was named **ASTRA**, an acronym for *Active Spine Technology for Robotic Agility*. The name emphasizes the central innovation of the platform: the integration of an actuated spine to enhance locomotion performance while also reflecting its intended characteristics of speed,

agility, and robustness. Beyond the acronym, *Astra* is derived from the Latin word for stars, evoking ambition and reliability, and historically associated with exploration and endurance; watching stars, you never get lost.

2 Hardware Design

2.1 General Overview

The quadruped has 15 motors which control 15 degrees of freedom: 3 per leg and 3 for the spine. It weighs **9.5 kg** and achieves a maximum walking speed of **4 cm/s** in its current hard-coded trot gait. It features an active spine with three degrees of freedom positioned between two rigid bodies, allowing for enhanced maneuverability and dynamic posture control. The actuation system is quasi-direct drive, providing backdrivability and precise torque control. The moments of inertia at the center of mass are the following:

Component	Value
I_{xx}	0.343
I_{yy}	0.077
I_{zz}	0.363
$I_{xy} = I_{yx}$	-5.317×10^{-5}
$I_{xz} = I_{zx}$	9.628×10^{-6}
$I_{yz} = I_{zy}$	-0.001

Table 2-1: Moments of inertia at the center of mass of the quadruped ($\text{kg} \cdot \text{m}^2$).

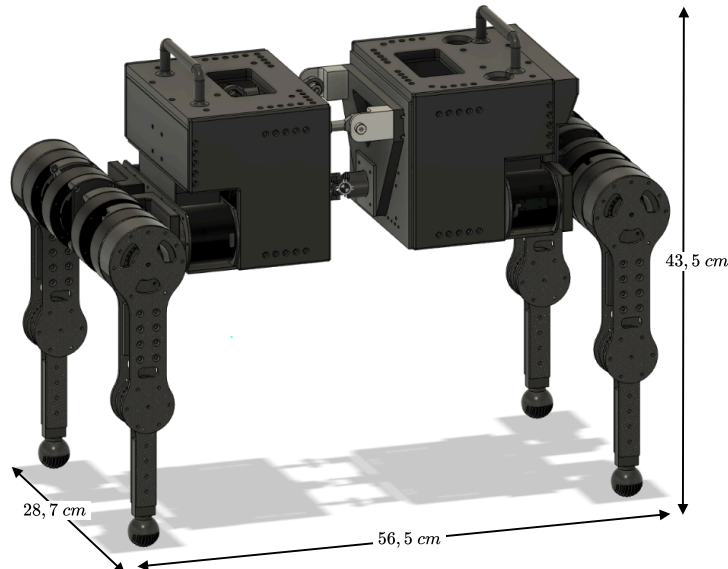


Figure 2-1: Rendering of the quadruped with CoG

Below is an overview of the connections and the placement of the major components in the quadruped. Figure 2-2 presents the main components of the system, their locations, and the communication network. The architecture can be compared to that of the human body: the Raspberry Pi 5 acts as the brain, the actuators (M in the figure) serve as the muscles, and everything in between functions as the nervous system. The STM32 microcontrollers operate only as bridges, transmitting data between the actuators and the Raspberry Pi. Communication between the Pi and the STM32 boards is established via USB, while the STM32 and the actuators communicate through CAN. The RoboMaster module (PDU in the diagram) serves as a power distribution unit: it is connected directly to the battery and provides the 24 V supply to the system. However, the Raspberry Pi 5 is powered by its own dedicated power bank, which makes it possible to include a shutdown button on the quadruped that cuts power to all actuators without interrupting the Pi.

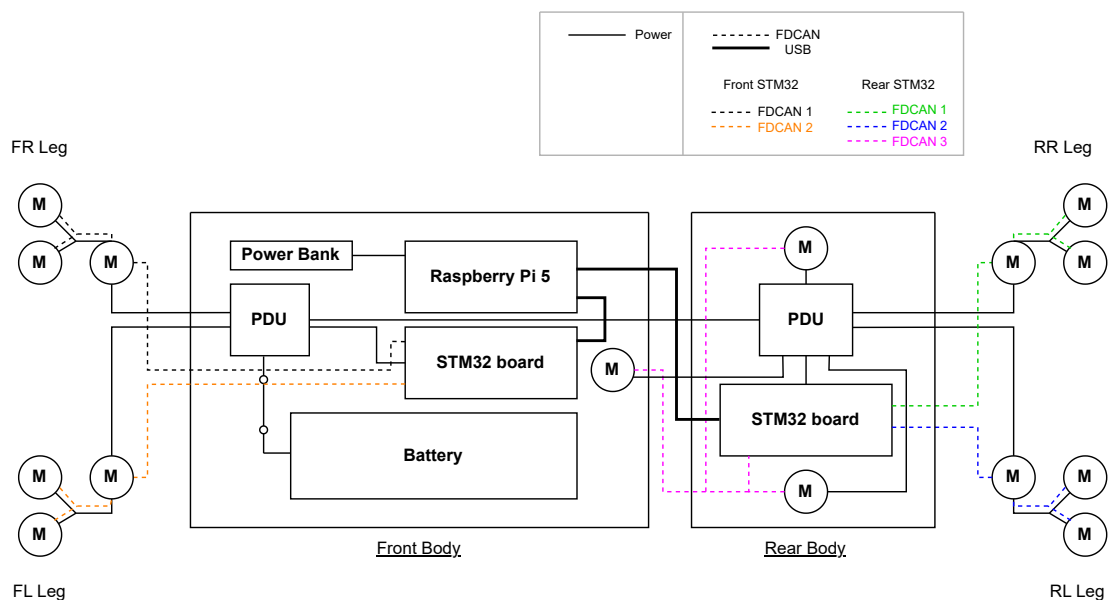


Figure 2-2: General electro-mechanical diagram

The naming convention used is shown in the Figure 2-3 where the actuators are in white:

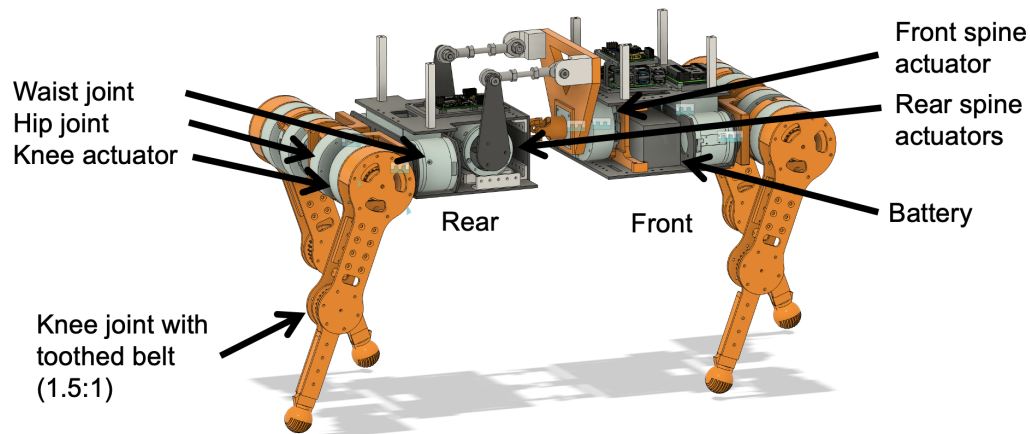


Figure 2-3: Naming convention

2.2 Actuator Choice

2.2.1 Actuator Choice

The selection of actuators is a critical design step in robotics, as it directly affects performance, weight, efficiency, and controllability. Several types of actuators are commonly employed in robotic systems:

- **Hydraulic actuators**: widely used in large-scale robots and industrial machinery for their extremely high power density and force output. They provide smooth motion and high stiffness but require bulky pumps, hoses, and valves, leading to higher system weight, increased maintenance demands, and reduced portability. Their use is therefore limited in small to medium-sized mobile robots.
- **Pneumatic actuators**: lightweight and inexpensive, capable of producing rapid motions. However, they suffer from low control precision, compressibility of air (which introduces compliance), and the need for external compressors, making them less suitable for quadrupeds where accurate torque and position control are required.
- **Series elastic actuators (SEA)**: these combine a motor with an elastic element to improve shock tolerance and energy storage. SEAs are well suited for safe interaction with humans and for legged locomotion with large impacts, but the addition of elastic elements reduces bandwidth and increases control complexity.
- **Electrical direct-drive actuators**: provide precise torque control and backdrivability by coupling the motor shaft directly to the load without high-ratio gearing. They offer high transparency and controllability but typically require very large, heavy motors to generate sufficient torque for legged robotics.
- **Quasi-direct drive actuators (QDD)**: a compromise between direct drive and geared solutions. They use low gear ratios (5:1 15:1), which maintain low reflected inertia and high

backdrivability while still amplifying torque to usable levels. QDD systems are increasingly popular in state-of-the-art quadrupeds because they combine robustness, efficiency, and precise force control while keeping the mass of the actuator moderate.

For this project, electrical actuators were selected, as they are more compact, energy-efficient, and easier to integrate than hydraulic or pneumatic systems in a lightweight quadruped. Within electrical actuators, two main categories are relevant: direct-drive (DD) and quasi-direct drive (QDD).

In a direct-drive configuration, the motor shaft is coupled directly to the joint without gear reduction. This yields maximum transparency, perfect backdrivability, and eliminates backlash or compliance introduced by transmission elements. However, achieving the torque levels required for quadruped knees with DD alone demands large, heavy motors, which increases overall robot mass and reduces efficiency.

Quasi-direct drive actuators address this limitation by introducing a small gear reduction, typically in the range of 5:1 to 15:1. The gear amplifies torque while keeping reflected inertia low enough to preserve backdrivability and force controllability. Compared to high-ratio geared actuators (e.g. 100:1 harmonic drives), QDD maintains nearly the same transparency as DD but allows the use of smaller, lighter, and more efficient motors. This design philosophy has become the standard in modern quadrupeds such as the MIT Mini Cheetah.

In the present quadruped, a mixed approach was adopted. The waist and hip joints are implemented in direct drive, as their torque requirements can be met by the selected motors without gearing. By contrast, the knee joint requires significantly higher torque, for which a 1.5:1 belt transmission was introduced. This classifies the knee as a quasi-direct drive joint, while the waist and hip remain purely direct drive.

The actuators chosen are Damiao brushless DC motors, which integrate both the motor and the driver on a single board. This integration eliminates the need for external driver boards: the motors expose a CAN interface that accepts predefined commands (enable, disable, set zero, request position, MIT-mode torque control, etc.). Consequently, the control architecture only needs to construct and transmit CAN messages, while all low-level motor control tasks such as commutation, current regulation, and protection are handled internally by the integrated driver. This feature drastically simplifies the electronics design and improves system reliability.

2.3 Electronics Hardware

2.3.1 Electronics choice

Different hardware architectures were considered for the electronic integration of the quadruped. A first option would have been to connect the actuators directly to the main computer (Raspberry Pi 5), without the use of intermediate microcontrollers. This approach

simplifies the hardware and reduces the number of boards, but it places the entire low-level motor communication and protocol handling burden on the host computer. In practice, this means that the host must handle all real-time CAN responsibilities itself, including bit timing, arbitration, retransmissions, acknowledgments, and DLC management. Such a setup would also require additional hardware, typically USB-to-CAN or PCIe-CAN adapters, which increase cost, add failure points, and reduce portability. Furthermore, the absence of an intermediate abstraction layer makes the architecture less modular: if the central computer is replaced, the low-level drivers and CAN interfaces would need to be reimplemented and validated.

A second possibility was to rely on a more powerful onboard computer instead of a Raspberry Pi, still connected directly to the actuators without microcontrollers. This solution would make it possible to run computationally intensive algorithms or vision processing locally, but at the cost of higher weight, larger volume and increased power consumption. Moreover, the same integration problem remains: direct CAN connection to the host still requires external adapters and specialized drivers, which complicates deployment and reduces flexibility.

For this project, a hybrid architecture was therefore selected, combining a Raspberry Pi 5 as the high-level control unit with two STM32 microcontrollers acting as communication bridges. The Raspberry Pi formats and sends the CAN payloads via USB, while the STM32 boards absorb the real-time responsibilities of the FDCAN layer. By offloading arbitration, interrupt-driven transmission scheduling, and low-level protocol compliance to the STM32s, the Raspberry Pi operates as a nonreal-time publisher of high-level commands. This modular approach provides several advantages. First, it allows the Raspberry Pi to be replaced seamlessly by any other computer equipped with a USB port, without requiring modifications to the motor communication layer. This means that future upgrades to a more powerful onboard computer can be implemented immediately by copying the ROS 2 nodes, with no changes to the underlying actuator communication. Second, it simplifies cabling: instead of routing three separate CAN buses along the robot body, only two USB connections are needed, one to the front STM32 and one to the rear, reducing wiring complexity, potential failure points, and overall assembly time.

The choice of CAN as the communication protocol between STM32s and actuators is justified by its robustness and performance. CAN is widely adopted in automotive and industrial applications due to its noise resistance, fault tolerance, and deterministic arbitration, making it well suited for long cable runs in electrically noisy environments. Furthermore, it supports multiple actuators on the same bus without point-to-point wiring, thereby reducing both cost and complexity. Although the inclusion of STM32 boards increases hardware count, their benefits in modularity, future scalability, and independence from specialized USB-to-CAN adapters clearly outweigh the added cost.

In summary, the selected architecture strikes a balance between performance, modularity, and maintainability. By combining a Raspberry Pi with STM32-based CAN bridges, the system remains flexible, portable, and robust, while also ensuring straightforward integration of future high-performance onboard computers for advanced control algorithms.

Criterion	Direct Computer-to-CAN	Raspberry Pi + STM32 Bridge (selected)
Hardware complexity	Fewest boards, but requires USB-to-CAN or PCIe-CAN adapters	Moderate: one Pi + two STM32 boards, no external adapters needed
Real-time CAN handling	Host must manage bit timing, arbitration, ACK, retransmission	Offloaded to STM32, host runs nonreal-time ROS 2 only
Modularity / portability	Low: changing host requires new drivers and reconfiguration	High: any USB-capable computer can replace the Pi instantly
Computational capability	Limited by Pi or by chosen host	Flexible: Pi today, easily swappable for a high-performance computer tomorrow
Weight / volume	Low	Low to moderate
Cost	High (due to CAN adapters)	Moderate (STM32 cost < adapters, Pi inexpensive)
Robustness in embedded use	Sensitive to adapter drivers	Robust, distributed, widely tested MCUs in robotics

Table 2-2: Comparison of electronic architectures for quadruped integration.

2.3.2 Electronics Architecture

The Raspberry Pi acts as the brain of the quadruped, while the STM32 boards serve only as bridges between the 15 actuators and the Pi. The Pi generates and sends a USB message to the STM32, which then forwards the relevant part of the message to the specified FDCAN bus. For simplicity, the frame formats of both USB and FDCAN are constructed on the Pi; the STM32 receives the instruction indicating to which leg and which motor the CAN message should be forwarded. In the following two subsections, the format of each protocol is explained in detail. Figure 2-4 presents a detailed overview of the communication architecture, showing the USB ports, the FDCAN buses, and the actuator IDs.

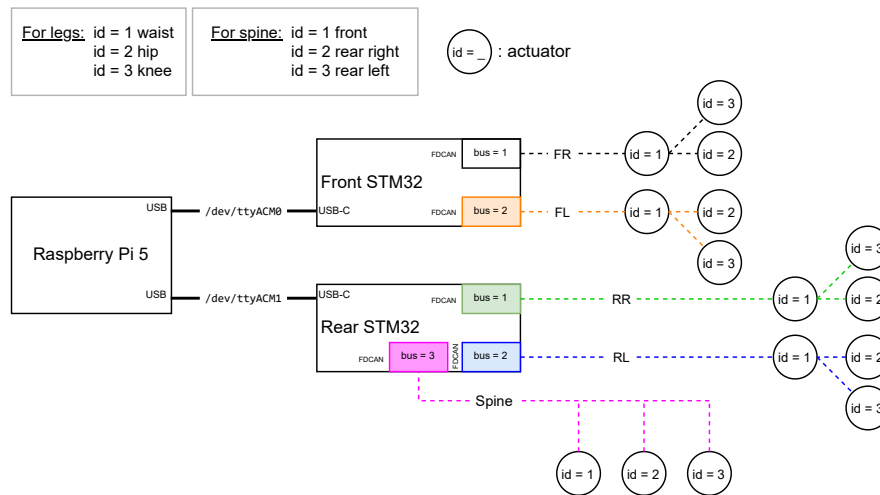


Figure 2-4: Overview of communications with usb ports, can buses and can ids

2.3.3 Communication frequency

The STM32H723VGT6 microcontroller is mounted on a custom board provided by Damiao Technology along with the motors. I used the three available CAN-FD ports and the USB-C interface for communication. Before defining the structure of the communication protocols, I first determined the maximum frequency at which messages could be reliably transmitted through the system.

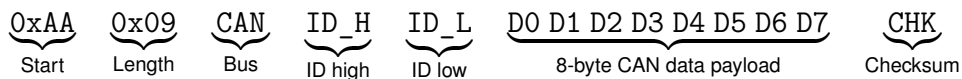
Device	Interface	Max Frequency / Baudrate	Source
Raspberry Pi 5	USB 3.0	5 Gbps	[Ras-2023]
STM32H723	FDCAN	8 Mbps (CAN FD)	[ST-2023a]
	USB (HS)	480 Mbps	[ST-2023a] Section 3.43
DM Board	CAN FD	5 Mbps	DM Board PDF, p.2
DM Motors	CAN	1 Mbps	Motor PDF, CAN@1Mbps

Table 2-3: Maximum communication frequencies of all devices in the system

Therefore, the maximum frequency is $f = 1 \text{ MBps} = 125 \text{ Byte/ms}$

2.3.4 USB communication protocol

The USB communication is used between the Raspberry Pi 5 and the two STM32 boards. Each message is 14 bytes long and contains all the information required to control the actuators. The structure of the message is the following:



Byte	Index	Field	Meaning
0		0xAA	Start byte
1		0x09	Payload length (always 9)
2		CAN bus	1 = FDCAN1, 2 = FDCAN2, 3 = FDCAN3
3-4		CAN ID	11-bit motor ID (big-endian)
5-12		Data[0-7]	8-byte CAN payload
13		Checksum	Sum of bytes 212 modulo 256

Table 2-4: Structure of the USB-to-CAN message format

2.3.5 FDCAN communication protocol

The standard CAN FD data frame is the following:

Figure 1. Standard CAN-FD frame

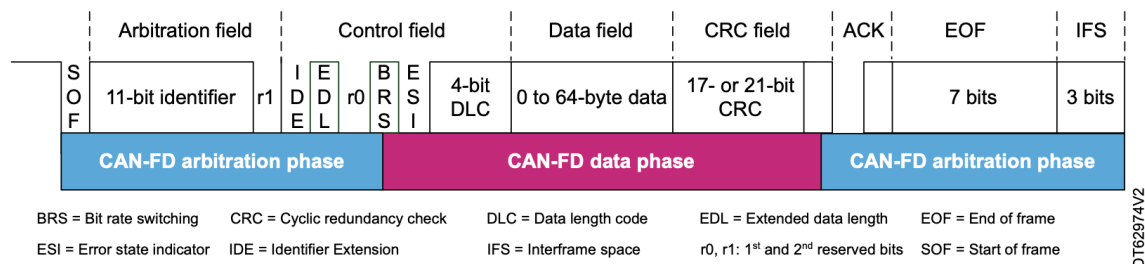


Figure 2-5: Standard CAN-FD frame (when not specified bit = 1) from [ST-2021]

The three fields that I can modify according to my necessities are: the 11-bit identifier, the 4-bit DLC and the data field.

- **11-bit identifier:** it is the can-ID (Figure 2-6)
- **4-bit DLC:** size of the data is 8 byte (imposed by DAMIAO Motors)
- **Data field:** 8 bytes which depend on the command, see next subsection (imposed by DAMIAO Motors)

Once the CAN-FD frame is defined, I had to configure the parameters of the FDCAN accordingly:

▼ Basic Parameters	
Frame Format	Classic mode
Mode	Normal mode
Auto Retransmission	Enable
Transmit Pause	Disable
Protocol Exception	Disable
Nominal Sync Jump Width	6
Data Prescaler	3
Data Sync Jump Width	5
Data Time Seg1	10
Data Time Seg2	5
Message Ram Offset	0
Std Filters Nbr	1
Ext Filters Nbr	0
Rx Fifo0 Elmts Nbr	1
Rx Fifo0 Elmt Size	8 bytes data field
Rx Fifo1 Elmts Nbr	0
Rx Fifo1 Elmt Size	8 bytes data field
Rx Buffers Nbr	0
Rx Buffer Size	8 bytes data field
Tx Events Nbr	0
Tx Buffers Nbr	0
Tx Fifo Queue Elmts Nbr	1
Tx Fifo Queue Mode	FIFO mode
Tx Elmt Size	8 bytes data field

Figure 2-6: *FDCAN setup: TX/RX on STM32*

▼ Clock Calibration Unit	
Clock Calibration	Disable
▼ Bit Timings Parameters	
Nominal Prescaler	1
Nominal Time Quantum	20.83333333333332 ns
Nominal Time Seg1	32
Nominal Time Seg2	15
Nominal Time for one Bit	1000 ns
Nominal Baud Rate	1000000 bit/s

Figure 2-7: *FDCAN setup: Bit Timing*

I recommend using this bit timing calculator for FDCAN to calculate these parameters. (a more detailed explanation can be found in this tutorial) They were calculated with the following characteristics:

- frequency = 48 000 kHz
- tolerance = 4 587 Ppm
- node delay = 250 Ns

The **Message Ram Offset** is calculated this way:

Table 4. “Element” size number depending on data field range

Data field (bytes)	Element size (RAM words)
0 to <u>8</u>	<u>4</u>
12	5
16	6
20	7
24	8
32	10
48	14
64	18

Figure 2-8: *Data field in equivalent RAM words, [ST-2021]*

Basic Parameters	
Frame Format	Classic mode
Mode	Normal mode
Auto Retransmission	Enable
Transmit Pause	Disable
Protocol Exception	Disable
Nominal Sync Jump Width	6
Data Prescaler	3
Data Sync Jump Width	5
Data Time Seg1	10
Data Time Seg2	5
Message Ram Offset	0
Std Filters Nbr	1 1 word
Ext Filters Nbr	0
Rx Fifo0 Elmts Nbr	1 4 words
Rx Fifo0 Elmt Size	8 bytes data field
Rx Fifo1 Elmts Nbr	0
Rx Fifo1 Elmt Size	8 bytes data field
Rx Buffers Nbr	0
Rx Buffer Size	8 bytes data field
Tx Events Nbr	0
Tx Buffers Nbr	0
Tx Fifo Queue Elmts Nbr	1 4 words
Tx Fifo Queue Mode	FIFO mode
Tx Elmt Size	8 bytes data field

Figure 2-9: Parameters for RAM Offset

Figure 4. CAN message RAM mapping

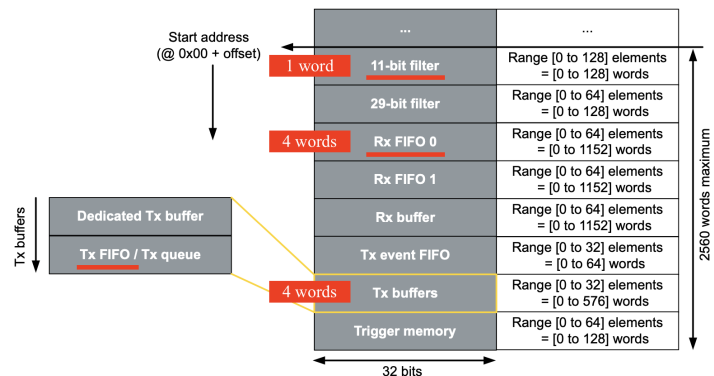


Figure 2-10: FDCAN Message RAM Offset

From which follows:

- FDCAN 1: Message Ram Offset = 0
- FDCAN 2: Message Ram Offset = 9
- FDCAN 3: Message Ram Offset = 18

In other words, for each CAN bus I will use 9 words in my RAM: so I allocate the first 9 words (0 to 8) to bus 1, the second 9 words (from 9 to 17) to bus 2, the third 9 words (from 18 to 26) to bus 3.

2.3.6 Damiao motor commands

As the actuators are bought, the commands are coded in the firmware and are predefined by the motor provider. For simplicity, I summarized the data field of each command in the following table:

Command	Data Field (Bytes 07)	Description
enable	[0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFC]	Enable motor
disable	[0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFD]	Disable motor
set_zero_position	[0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFE]	Set current position to zero
request	[0x00, 0x00, 0x00, 0x00, regID, 0x33, ID_MSB, ID_LSB]	Read position register
mit	[QH, QL, dQH, dQL_KpH, KpL, KdH, KdL_TauH, TauL]	MIT-mode command

Table 2-5: Byte-wise structure of the 8-byte CAN data fields

The **request command** will make the motor send back an 8-byte CAN frame in which bytes 47 contain the requested value encoded as a 32-bit IEEE 754 float (little-endian). The position is then decoded in the STM32 and sent to the pi as a raw 4-byte binary float.

The **MIT mode command** sends five control values — desired position q , velocity $\dot{q}$, torque τ , proportional gain K_p , and derivative gain K_d — packed into an 8-byte CAN data frame. Each value is first clamped to its physical range, then quantised:

$$\text{uint} = \text{round} \left(\frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot (2^b - 1) \right)$$

where b is the number of allocated bits. The result is packed in big-endian order using a nibble-stuffed layout as shown in the table below:

Byte	Bits	Contents
0	Q[15:8]	Position high byte
1	Q[7:0]	Position low byte
2	dQ[11:4]	Velocity high bits
3	dQ[3:0] $\ll$ 4 Kp[11:8]	4 bits of velocity + 4 high bits of K_p
4	Kp[7:0]	K_p low byte
5	Kd[11:4]	K_d high bits
6	Kd[3:0] $\ll$ 4 τ [11:8]	4 bits of K_d + 4 high bits of torque
7	τ [7:0]	Torque low byte

Table 2-6: MIT-mode 8-byte CAN data layout (big-endian, nibble-stuffed)

2.4 Mechanical Design

2.4.1 Frame and supports

The frame was designed using 2D plates and aluminum bars to keep it as light, simple, and cost-efficient as possible. Two materials were considered for the plates: CFRP and aluminum. While aluminum plates are cheaper to manufacture (e.g., via waterjet cutting), they are nearly twice as heavy, with densities of $\rho_{\text{CFRP}} = 1.55 \text{ g/cm}^3$ and $\rho_{\text{Al}} = 2.7 \text{ g/cm}^3$. Given the importance of reducing inertia in a quadraped, the lighter CFRP plates were selected despite their higher cost.

For the electronic components, the main design criteria were weight, space efficiency, and cost. To meet these requirements, a custom in-house 3D-printed support was developed, which allowed a compact and lightweight integration of the boards.

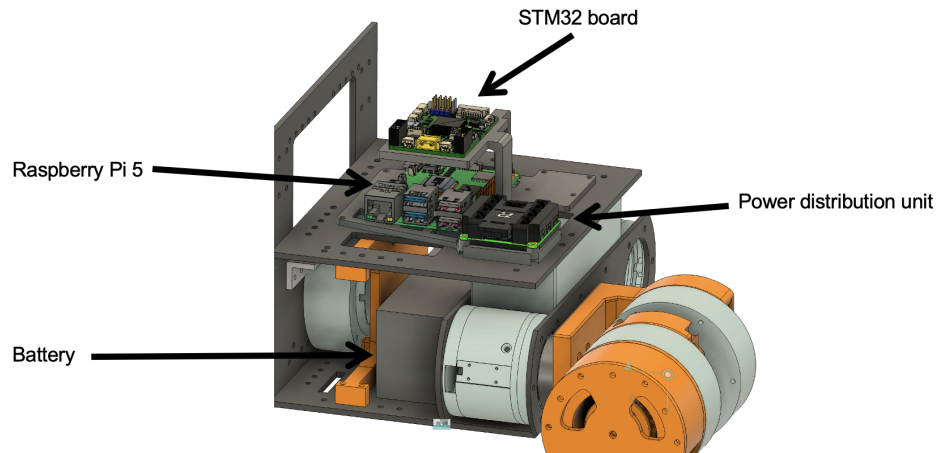


Figure 2-11: Electronics 3D printed front support

Finally, 3D-printed cover plates were added to enclose the structure, shielding the electronics, battery, and wiring from external impacts.

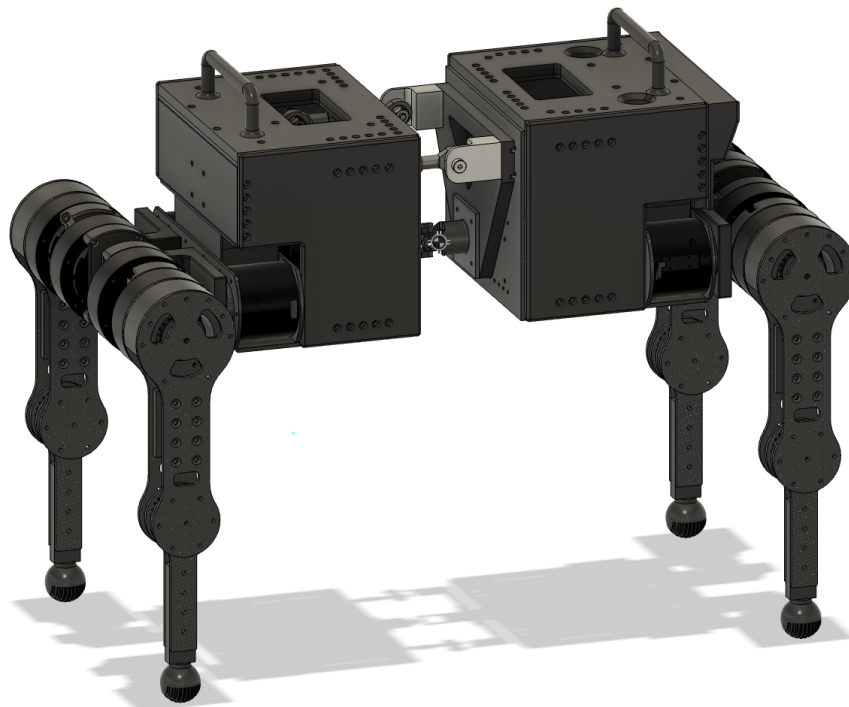


Figure 2-12: Rendering of the quadruped

2.4.2 Leg Design

The first step is to calculate the minimum required torque for each actuator. The chosen scenario is a static trot posture, in which the quadruped balances on two diagonal legs. It is common practice to compute the torque requirements under static conditions and then apply a safety margin typically 25%- to account for dynamic effects. Below, the two free body diagrams (FBD) to calculate the torque of each actuator.

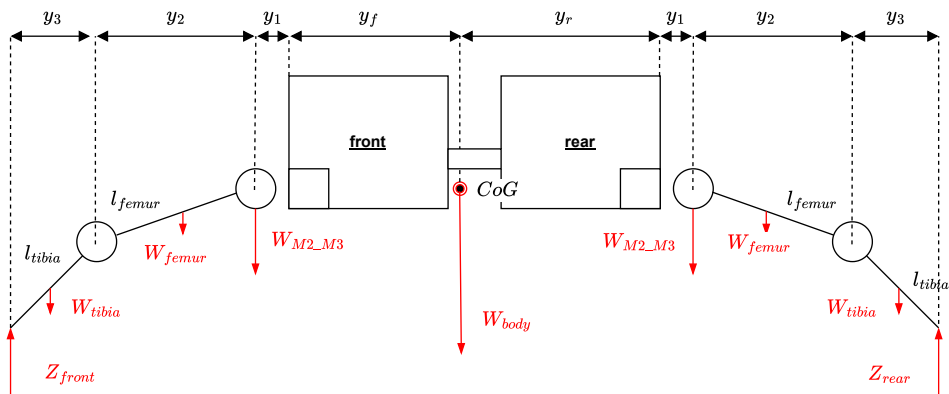


Figure 2-13: FBD for torque calculations in yz plane view

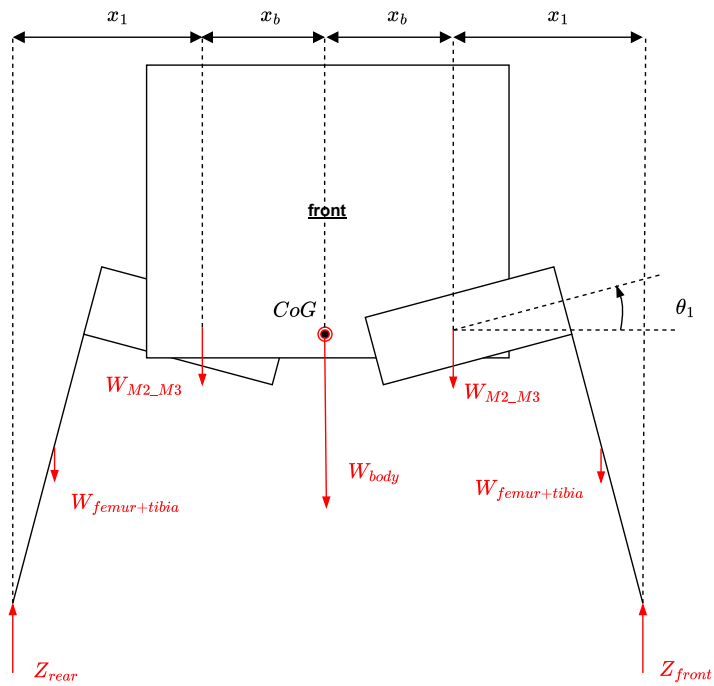


Figure 2-14: FBD for torque calculations in xz plane view

Which result in the following equations:

$$\begin{aligned}
Z_{\text{rear}} = & - \left(-W_{\text{tibia}} \cdot \left(\frac{y_3}{2} \right) - W_{\text{femur}} \cdot \left(\frac{y_3 + y_2}{2} \right) - W_{\text{M3_M2}} \cdot (y_3 + y_2) \right. \\
& - W_{\text{body}} \cdot (y_3 + y_2 + y_1 + y_f) - W_{\text{M3_M2}} \cdot (y_3 + y_2 + 2y_1 + Y) \\
& \left. - W_{\text{femur}} \cdot \left(y_3 + y_2 + 2y_1 + Y + \frac{y_2}{2} \right) - W_{\text{tibia}} \cdot \left(y_3 + y_2 + 2y_1 + Y + y_2 + \frac{y_3}{2} \right) \right) \\
& / (y_3 + y_2 + 2y_1 + Y + y_2 + y_3) \\
Z_{\text{front}} = & W_{\text{tot}} - Z_{\text{rear}} \\
T_1 = & -Z_{\text{front}} \cdot x_1 + (W_{\text{femur}} + W_{\text{tibia}}) \cdot \frac{3}{4} \cdot x_1 - (W_{\text{femur}} + W_{\text{tibia}}) \cdot \left(\frac{3}{4} \cdot x_1 + 2x_b \right) \\
& - W_{\text{body}} \cdot x_b - W_{\text{M3_M2}} \cdot 2x_b + Z_{\text{rear}} \cdot (2x_b + x_1) \\
T_2 = & Z_{\text{front}} \cdot (y_3 + y_2) - W_{\text{tibia}} \cdot \left(\frac{y_3}{2} + y_2 \right) - W_{\text{femur}} \cdot \frac{y_2}{2} + W_{\text{body}} \cdot (y_1 + y_f) \\
& + W_{\text{M3_M2}} \cdot (2y_1 + Y) + W_{\text{femur}} \cdot \left(2y_1 + Y + \frac{y_2}{2} \right) + W_{\text{tibia}} \cdot \left(2y_1 + Y + y_2 + \frac{y_3}{2} \right) \\
& - Z_{\text{rear}} \cdot (2y_1 + Y + y_2 + y_3) \\
T_3 = & Z_{\text{front}} \cdot y_3 - W_{\text{tibia}} \cdot \frac{y_3}{2} + W_{\text{femur}} \cdot \frac{y_2}{2} + W_{\text{M3_M2}} \cdot y_2 + W_{\text{body}} \cdot (2y_2 + y_1 + y_f) \\
& + W_{\text{M3_M2}} \cdot (y_2 + 2y_1 + Y) + W_{\text{femur}} \cdot \left(y_2 + 2y_1 + Y + \frac{y_2}{2} \right) \\
& + W_{\text{tibia}} \cdot \left(2y_2 + 2y_1 + Y + \frac{y_3}{2} \right) - Z_{\text{rear}} \cdot (2y_2 + 2y_1 + Y + y_3)
\end{aligned}$$

The numerical results are presented below as a plot for the waist joint and as graphs for the hip and knee joint. The torque needed for each actuator is the following: 3.1 Nm for waist, 5.4 Nm for hip, 6.3 Nm for knee (considering the 1.5 gear ratio).

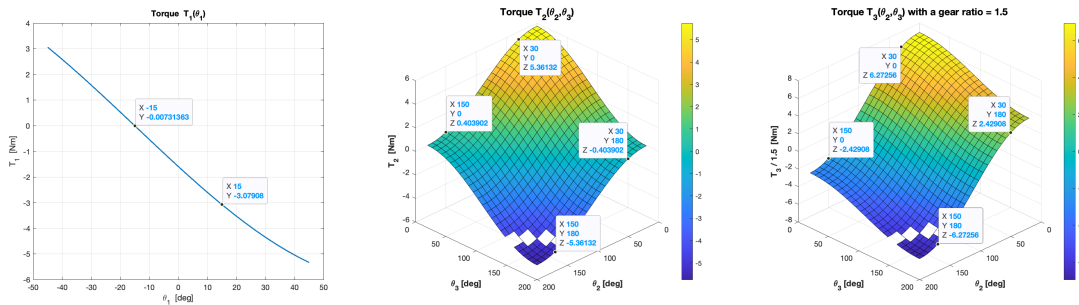


Figure 2-15: Minimum torque results for legs actuators

From the specifications of the DAMIAO motors available, it follows that the DM6006 has to be chosen for both the hip and the knee. For the knee joint, a belt is added to the design with a gear ratio of 1.5 to make the torque output even higher.

Motor type	Rated torque	Peak torque	Rated current	Peak current	Rated speed
DM-J4310-2EC	3 Nm	7 Nm	2.5 A	7.5 A	120 rpm
DM-J6006-2EC	4 Nm	12 Nm	4 A	13 A	150 rpm

Table 2-7: Motor specifications from manufacturer datasheets

After selecting the actuators for each joint, the leg was designed using a combination of 3D-printed parts and CFRP plates. The resulting structure has a mass of 0.35 kg without actuators and 1.35 kg with actuators installed.

Part	Material	Mass [kg]
Waist to hip support	Tough PLA	0.073
Hip to knee support	Tough PLA	0.050
Gears for toothed belt	Tough PLA	0.031
Foot structure	Tough PLA	0.025
CFRP structure	CFRP	0.100

Table 2-8: Mass of major leg components

Figure 2-16 illustrates the leg components: actuators are shown in light blue, 3D-printed parts in white, and CFRP plates in black.

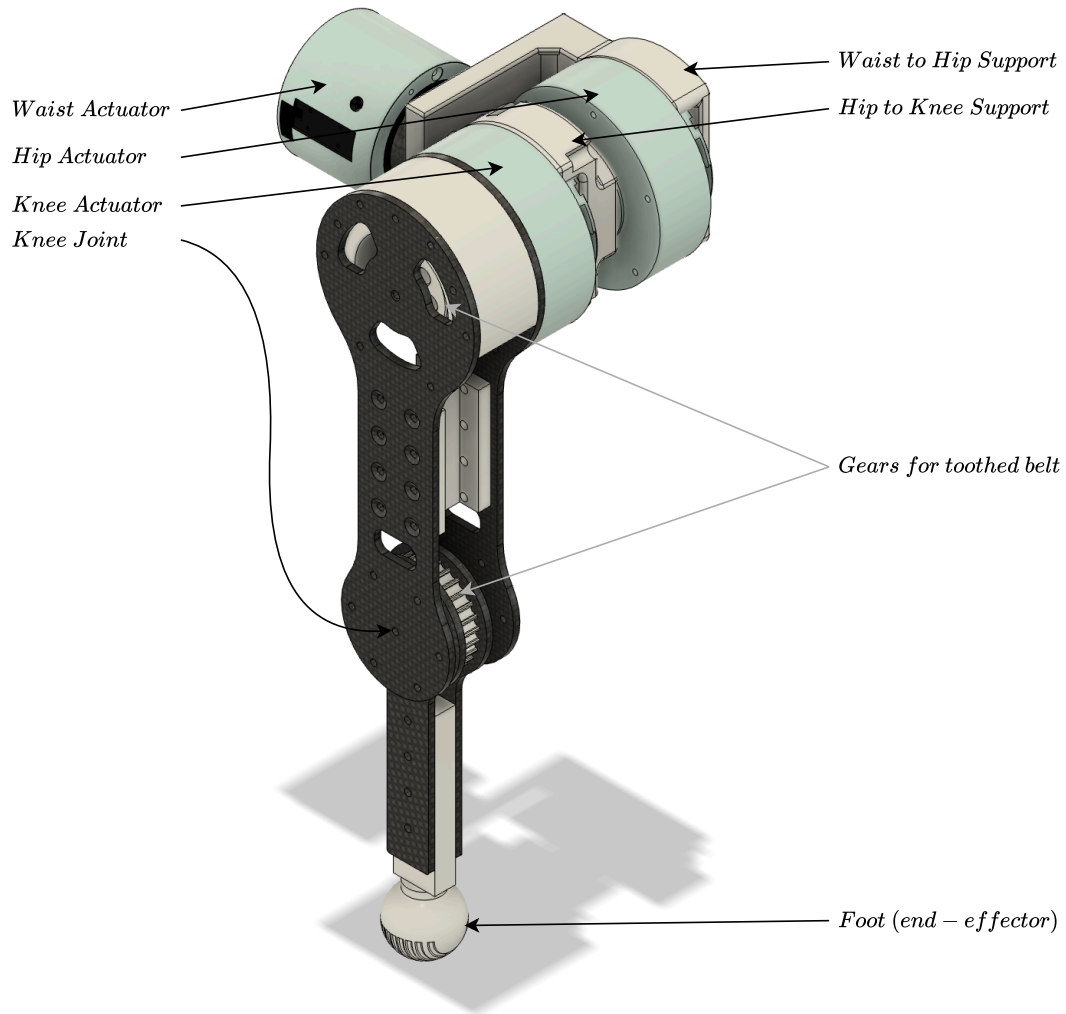


Figure 2-16: Leg design

2.4.3 Spine Design

Parallel design vs serial design

A parallel design was chosen over a serial one, as it integrates more naturally with the two-body structure of the quadruped. A serial design would have required splitting the chassis into additional segments, since controlling three degrees of freedom would necessitate three distinct bodies connected in series. By contrast, the parallel architecture achieves the same functionality within the existing two-body layout. Furthermore, the parallel design provides higher torque density and improved controllability, at the expense of a reduced range of motion. In this application, however, the available motion range is sufficient, making the trade-off acceptable. In addition, the closed-chain structure of the parallel mechanism increases stiffness and distributes loads across several links, which reduces local deformations and enhances durability under high forces. Locating the actuators closer to the pivot also helps reduce reflected inertia on the moving parts. The drawbacks are a higher sensitivity to manufacturing tolerances, more complex assembly, and a workspace

that can be more limited than in serial designs. Nevertheless, in this quadruped application these disadvantages remain acceptable, as the required workspace is modest and the benefits in torque capacity, rigidity, and controllability are decisive.

Building on this choice, the specific design is inspired by humanoid ankle mechanisms, which also exploit parallel architectures to combine high torque with compactness. In the quadruped, however, an additional degree of freedom was introduced, allowing the front and rear body segments to rotate in opposite directions about the y-axis.

Mechanical Integration

The mechanical integration then focused on three main criteria: minimizing compliance and deformation, avoiding significant increases in overall body length, and ensuring an equal weight distribution between front and rear so that the center of gravity remains as close as possible to the pivot point.

As with the leg design, the first step was to estimate the minimum torque requirements. The same scenario was assumed: a static trot posture in which two diagonal legs support the entire quadruped.

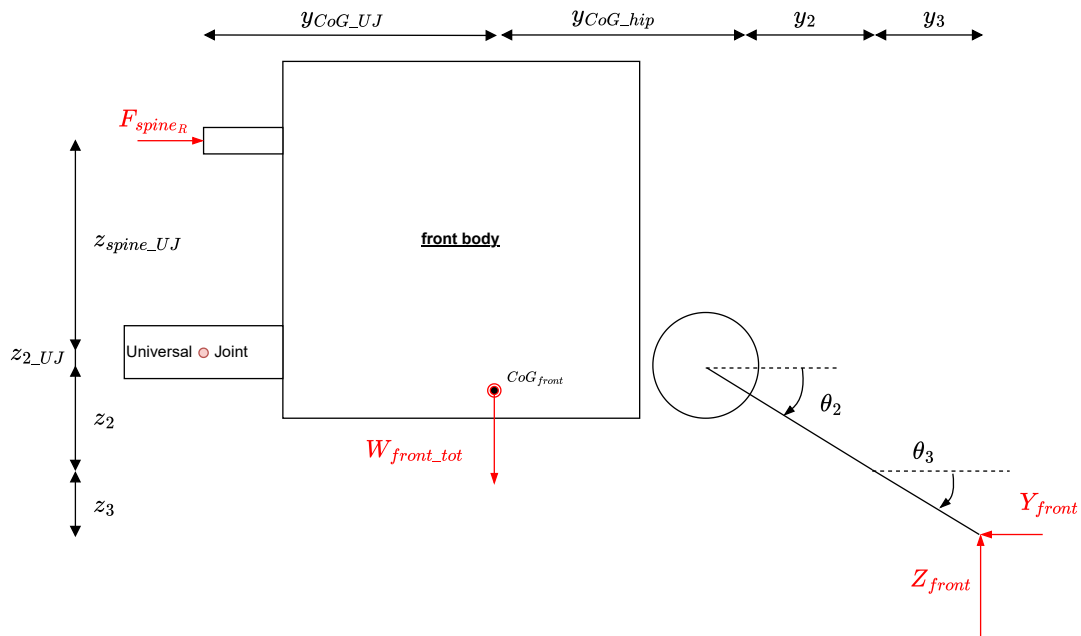


Figure 2-17: FBD for spinal torque calculation in yz plane

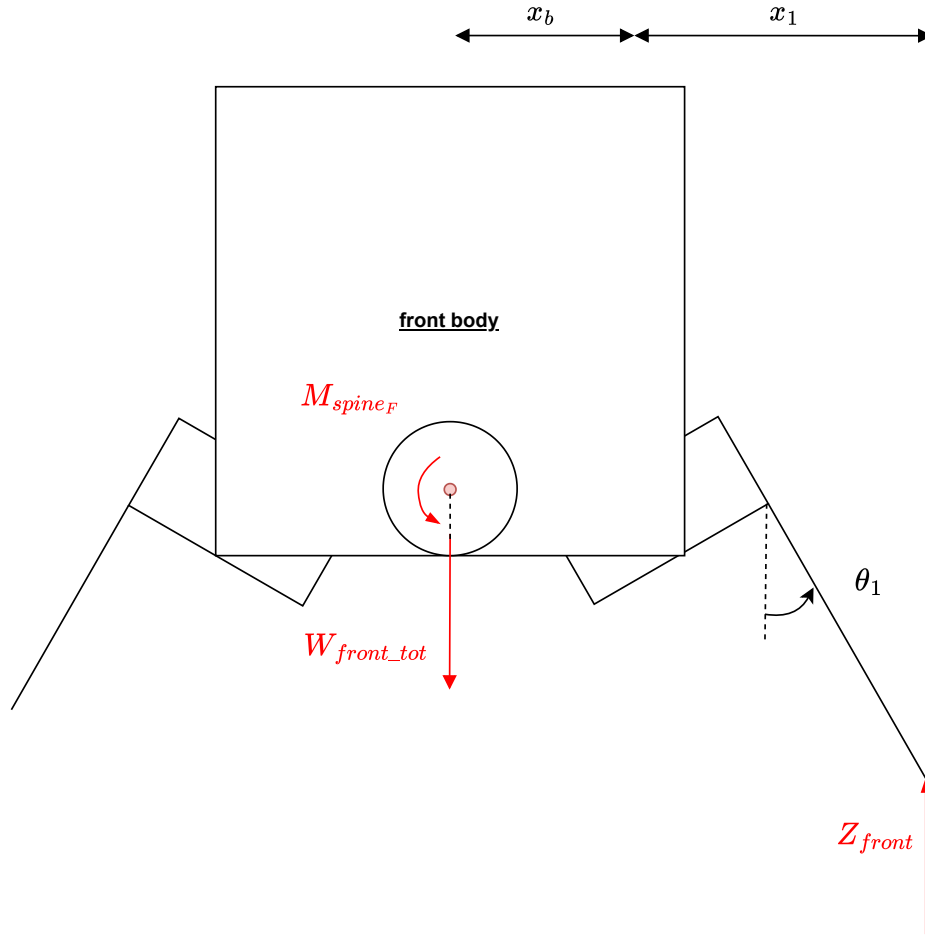


Figure 2-18: FBD for spinal torque calculation in xz plane

This leads to the following equations: for Figure 2-17, the moment equilibrium is taken at the universal joint (red point), whereas for Figure 2-18, the moment equilibrium is considered at the front actuator (red point).

$$F_{\text{spine},R} = \frac{-W_{\text{front,tot}} \cdot y_{\text{cog,uj}} + Z_{\text{front}} \cdot (y_{\text{cog,uj}} + y_{\text{cog,hip}} + y_2 + y_3)}{Z_{\text{spine,uj}} + Z_{2,\text{uj}} + Z_2 + Z_3}$$

$$T_{\text{spine,rear}} = \frac{F_{\text{spine},R}}{2} \cdot z_{\text{spine,uj}} \quad (\text{we divide by 2 to account for the two actuators})$$

$$T_{\text{spine,front}} = W_{\text{front,tot}} \cdot (x_b + x_1)$$

The results are presented as a plot for the front spine joint and as a graph for the rear spine joint. The required torque values are 10.0 Nm for the front spine actuator and 3.5 Nm for the rear spine actuator.

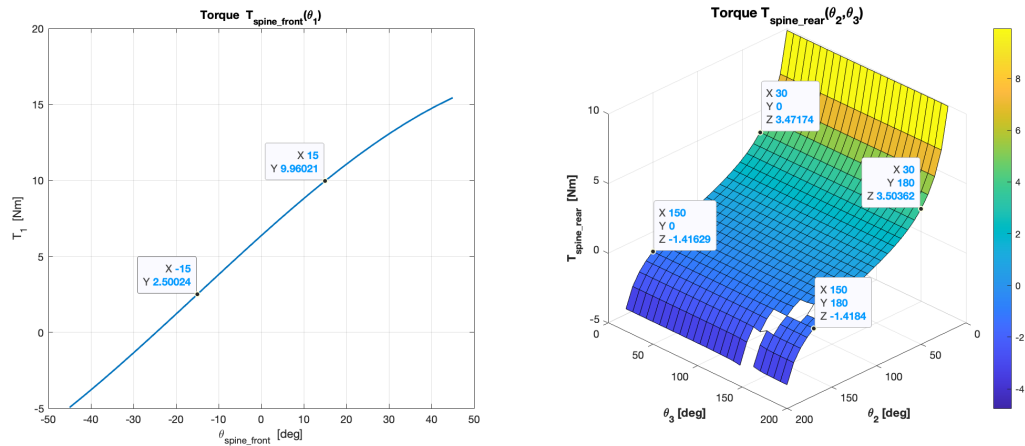


Figure 2-19: Minimum torque results for spine actuators

Final Design

The final design consists of 3D-printed parts, CFRP plates, steel bar with ball joints, and steel universal joint. The total body length is 56.5 cm, and the center of gravity is positioned close to the rotation center of the universal joint. The spine weighs 0.75 kg without actuators and 1.70 kg with actuators installed.

Part	Material	Mass [kg]
Front actuator support	Tough PLA	0.055
Spine support	Tough PLA	0.165
Rear actuator support	Tough PLA	0.055
Link	Aluminium	0.050g
Bar	CFRP	0.012
Universal Joint	Steel	0.220

Table 2-9: Mass of the major spine components

The spine support was designed to allow easy replacement with aluminum plates in case greater stiffness is required for future tests. In Figure 2-20 and Figure 2-21, the 3D-printed parts are shown in white, the actuators in light blue, the CFRP bar in black, and the link in grey.

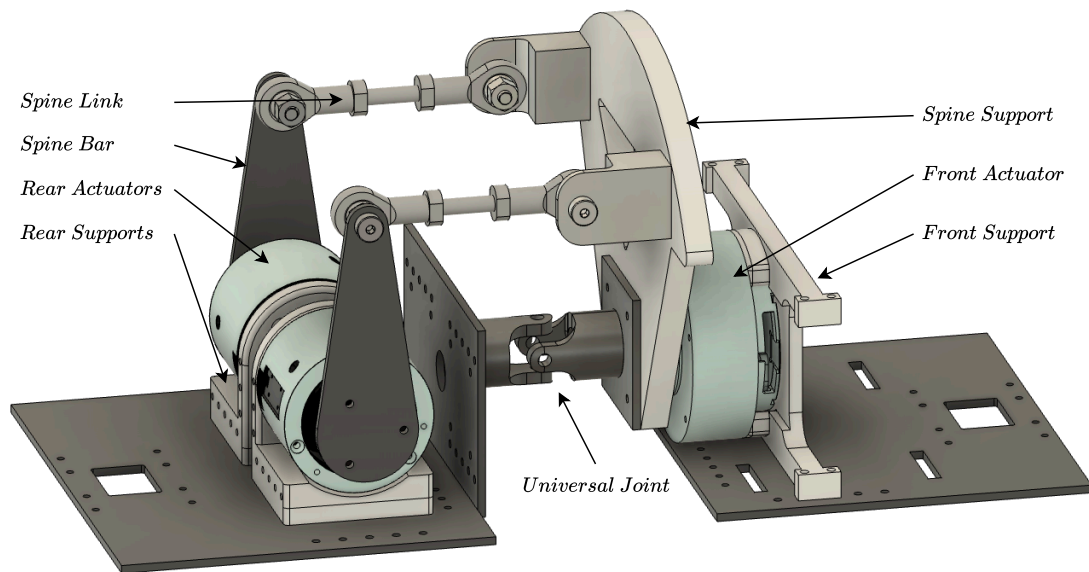


Figure 2-20: General view of the spine design.

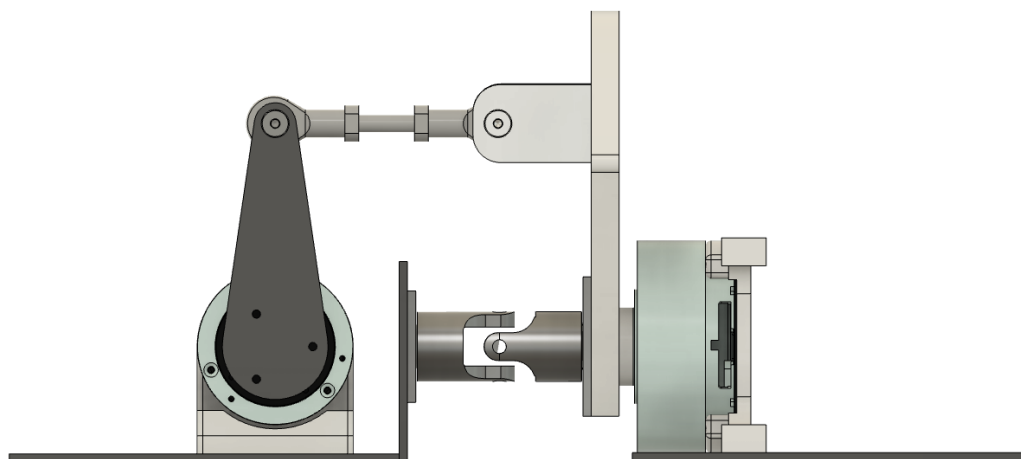


Figure 2-21: Side view of the spine design.

3 Control

3.1 Approach

Since the main objective of this thesis is to demonstrate that the proposed quadruped design is mechanically robust and controllable, the control algorithms implemented remain at a low level. Specifically:

- **Gait control:** a predefined closed-loop trot trajectory is executed using position control, with target joint angles computed via inverse kinematics.
- **Inclined surface balance:** the pitch angle of the spine is adjusted based on feedback from the IMU.

In the present work, the legs and the spine are controlled separately for the sake of simplicity. However, from a control perspective, they are part of a single coupled system: the spine motion directly influences stride length, energy efficiency, and balance, while the legs provide the necessary ground reaction forces to stabilize the body. Future work should therefore focus on integrated control strategies, where the legs and spine are optimized together. For example, a Model Predictive Controller (MPC) could include both the leg joints and the spine joints in its state vector, allowing the optimization to generate coordinated movements. Similarly, Central Pattern Generators (CPGs) or learning-based controllers could be extended to treat the spine as an integral component of locomotion rather than as a separate correction mechanism.

The following sections present the forward and inverse kinematics equations for both the legs and the spine. Subsequently, the formulation of the trot motion and the strategy for balance control are described in detail.

3.2 Inverse Kinematics and Forward Kinematics of the leg

In this project, forward kinematics were only used to check if the inverse kinematics were correct. The coordinate frame for each leg is defined with its origin at the center of the waist actuator's rotating output surface, which is directly screwed to the rest of the leg:

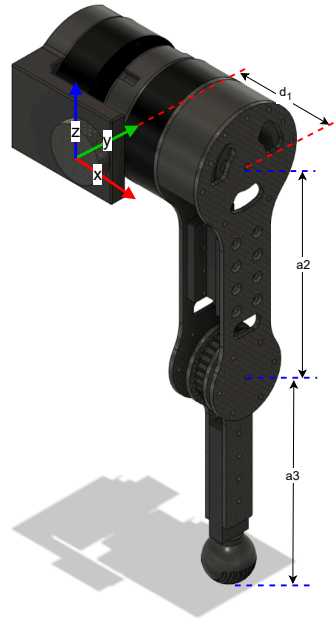


Figure 3-1: Origin of the frame on the leg in its initial position for IK and associated dimensions

The orientation of the coordinate frame of each leg is the following:

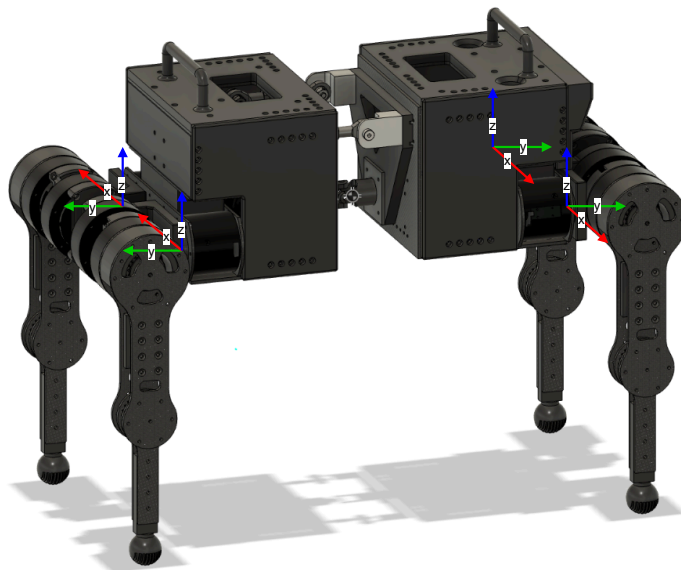


Figure 3-2: Orientation of the coordinate frames of the four legs

In this case, the parameters have the following values:

$d_1 = 5.5 \text{ cm}$ frame-to-end-effector x-distance

$d_2 = 6 \text{ cm}$ frame-to-end-effector y-distance

$a_2 = 14 \text{ cm}$ frame-to-knee joint z-distance

$a_3 = 14.1 \text{ cm}$ knee-to-end-effector z-distance

3.2.1 Forward Kinematics

In this project, forward kinematics were used primarily to validate the inverse kinematics. The idea was to compute the joint angles through IK for a desired end-effector position, and then apply FK to verify that the reconstructed position matches the desired one:

$$\text{FK}(\text{IK}(\text{des_pos}_{\text{end_effector}})) - \text{des_pos}_{\text{end_effector}} = 0.$$

Several approaches exist to formulate forward kinematics. The most traditional one is the DenavitHartenberg (DH) convention, which parameterizes each joint with four values and builds the transformation step by step. It is compact and widely used in robotics literature, but it can become cumbersome for complex mechanisms with non-standard joint layouts. A more modern and general approach is the Product of Exponentials (POE) formula, which expresses each joint motion as a matrix exponential of its screw axis. This method is well-suited to open kinematic chains. Finally, FK can also be implemented in a purely numerical form, where transformations are composed iteratively using homogeneous matrices or quaternions; this is straightforward to code but less compact analytically.

In this thesis, the POE formulation was adopted, as it provides a consistent and general framework that integrates well with the analytical IK derivation already used for the legs.

Forward Kinematics via Product of Exponentials

As the leg can be modelled as an open-kinematic chain, the forward kinematics are computed using the product of exponentials formula, which requires:

- **Definition of screw axes:** each joint axis must be expressed as a twist $[S_i]$ in 6D space (angular and linear velocity components).
- **Home configuration:** the end-effector pose M must be defined at the reference position ($\theta_i = 0$).
- **Joint angles:** the current configuration is parameterized by the joint values θ_i .

Hence, the forward kinematics are computed with:

$$T(\theta) = e^{[S_1]\theta_1} \cdot e^{[S_2]\theta_2} \cdot e^{[S_3]\theta_3} \cdot M$$

where:

- $[S_i] \in \mathbb{R}^{4 \times 4}$ are the screw axis matrices in space frame
- θ_i are the joint angles
- M is the home configuration of the end-effector

The home configuration matrix M is:

$$M = \begin{bmatrix} 1 & 0 & 0 & d_1 \\ 0 & 1 & 0 & d_2 \\ 0 & 0 & 1 & a_2 + a_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The screw axes in space frame are:

$$S_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T, \quad S_2 = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & -d_2 \end{bmatrix}^T$$

$$S_3 = \begin{bmatrix} 0 & 1 & 0 & d_2 & 0 & -(a_2 + a_3) \end{bmatrix}^T$$

After evaluating the full transformation matrix $T(\theta) \in SE(3)$, the position of the end-effector in 3D space is obtained from the translation part (last column):

$$\text{Position}_{\text{end-effector}} = \begin{bmatrix} T_{14} \\ T_{24} \\ T_{34} \end{bmatrix}$$

3.2.2 Inverse Kinematics

There are two main approaches to solving inverse kinematics: numerical and analytical. Numerical methods (e.g., NewtonRaphson, Jacobian-based solvers) are general and can handle almost any kinematic structure, even when the geometry is complex or redundant. They are also relatively easy to implement. However, they require iterative computation, which increases computational cost, and may converge slowly or get stuck in local minima if the initial guess is poor.

Analytical methods, on the other hand, exploit the geometry of the mechanism to derive closed-form equations for the joint angles. When such a solution exists, it provides exact results with very low computation time, which is particularly advantageous for real-time control. The downside is that analytical solutions are only possible for relatively simple kinematic chains and become cumbersome to derive as the number of joints and complexity of the geometry increases.

In this project, both methods were tested at the beginning. Ultimately, the analytical approach was chosen and the inverse kinematics function was hard-coded, in order to minimize computing time and ensure performance during real-time control.

For a given leg, let the desired foot position be (p_x, p_y, p_z) . For simplicity, the reference frame is aligned with the axes of the hip and knee actuators, introducing a translation of $+d_2$ along the y -axis. As a result, the transformed foot position becomes

$$p_{y,\text{new}} = p_y - d_2.$$

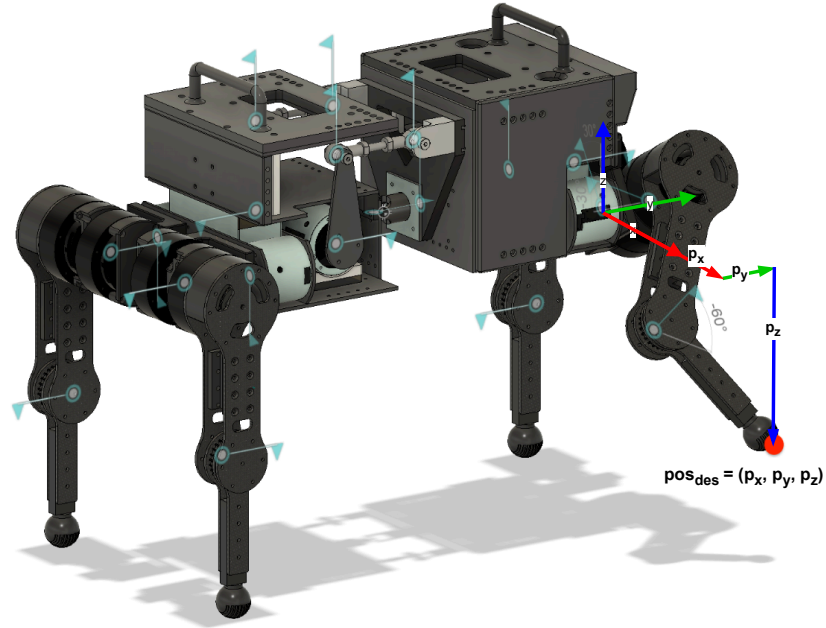


Figure 3-3: Quadruped with end-effector desired position for front right (FR) leg

We define:

$$r^2 = p_x^2 + p_z^2$$

$$\phi = \arctan\left(\frac{p_z}{p_x}\right), \quad \alpha = \arctan\left(\frac{\sqrt{r^2 - d_1^2}}{d_1}\right)$$

Then the hip (abduction) angle θ_1 is:

$$\theta_1 = |\phi| - |\alpha|$$

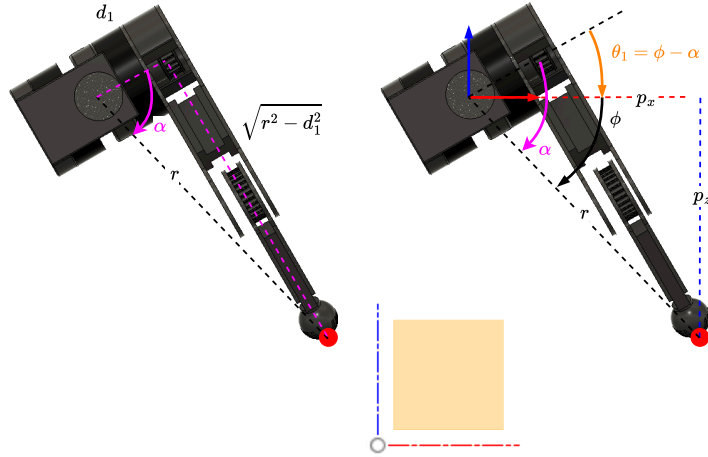


Figure 3-4: Front right leg view in x-z plane with end effector desire position

The knee angle is computed using the cosine law:

$$D = \frac{p_x^2 + p_y^2 + p_z^2 - d_1^2 - a_2^2 - a_3^2}{2a_2a_3}, \quad \theta_3 = \arctan\left(\frac{\sqrt{1 - D^2}}{D}\right)$$

Let $s_3 = \sin(\theta_3)$, $c_3 = \cos(\theta_3)$. Then the shoulder angle is:

$$\theta_2 = \arctan\left(\frac{p_y}{\sqrt{r^2 - d_1^2}}\right) - \arctan\left(\frac{a_3 s_3}{a_2 + a_3 c_3}\right)$$

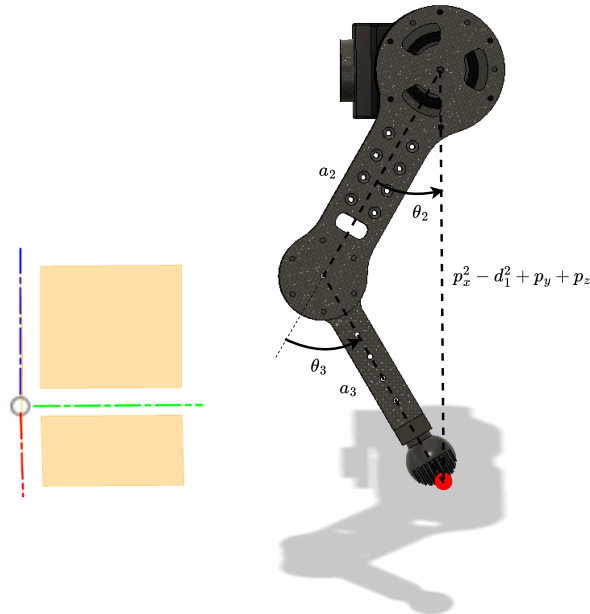


Figure 3-5: Front-right leg view in leg plane with end effector desire position

For legs (1, 2) (front left and rear right), the angles are negated:

$$\theta_{1_new} = -\theta_1, \theta_{2_new} = -\theta_2, \quad \theta_{3_new} = -\theta_3$$

Explanation for the negations:

The joint angle θ_1 is negated for leg 1 to produce a waist movement symmetric to that of leg 0. It is also negated for leg 2 because the rotational axis of its waist actuator, following the right-hand rule, is oriented opposite to that of leg 0. The angles θ_2 and θ_3 are negated for leg 1 because their actuator rotation directions are reversed compared to leg 0. For leg 2, θ_2 and θ_3 are negated to enforce a mirrored posture with respect to the front legs, such that the knee bends in the opposite direction.

3.3 Inverse Kinematics and Forward Kinematics of the spine

3.3.1 Inverse Kinematics

The spine is inspired by the ankle design of humanoids, it is a parallel kinematics mechanism. [Zho-2023] presents an extensive kinematic analysis of this design, and the inverse kinematics derivation is also detailed below. The frame is different than of the legs' inverse kinematics.

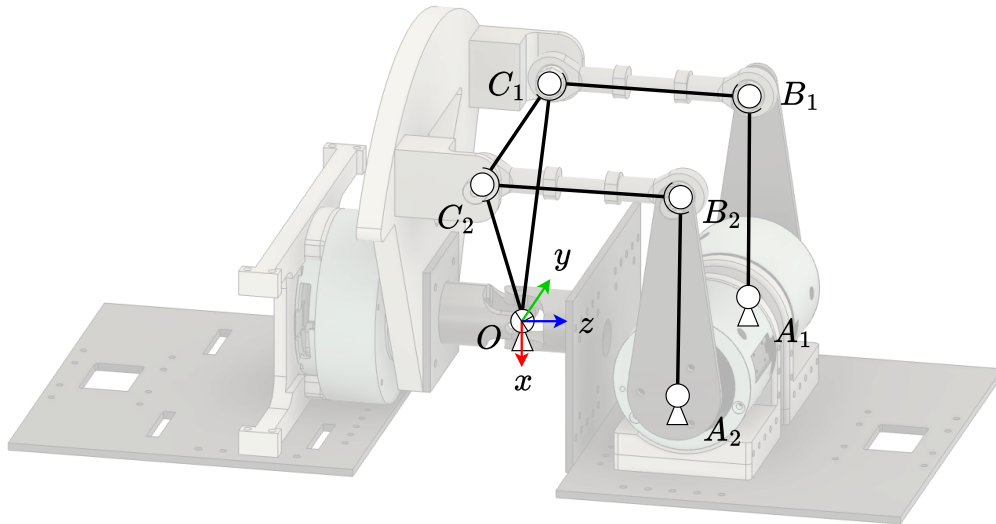


Figure 3-6: Spine diagram

It always holds true that:

$$\|\mathbf{r}_{C_i} - \mathbf{r}_{B_i}\| = l_{rod}, \quad \text{for } i = 1, 2,$$

Solving the above equations we get to: let theta be the motors' position,

$$\theta_i = \arcsin \left(\frac{bc + \sqrt{b^2c^2 - (a^2 + b^2)(c^2 - a^2)}}{a^2 + b^2} \right),$$

where,

$$\begin{aligned} a &= \{\mathbf{r}_{C_i} - \mathbf{r}_{A_i}\}_x, \\ b &= \{\mathbf{r}_{A_i} - \mathbf{r}_{C_i}\}_z, \\ c &= \frac{l_{rod}^2 - l_{bar}^2 - \|\mathbf{r}_{C_i} - \mathbf{r}_{A_i}\|^2}{2l_{bar}}, \end{aligned}$$

$$\begin{aligned} l_{bar} &= \|\mathbf{r}_{B_i} - \mathbf{r}_{A_i}\| = \{\mathbf{r}_O - \mathbf{r}_{C_i}\}_x = 80 \text{ mm}, \\ l_{rod} &= \|\mathbf{r}_{C_i} - \mathbf{r}_{B_i}\| = \{\mathbf{r}_O - \mathbf{r}_{A_i}\}_z = 80 \text{ mm}, \\ l_{spacing} &= \|\mathbf{r}_{A_1} - \mathbf{r}_{A_2}\| = \|\mathbf{r}_{B_1} - \mathbf{r}_{B_2}\| = \|\mathbf{r}_{C_1} - \mathbf{r}_{C_2}\| = 122 \text{ mm}, \end{aligned}$$

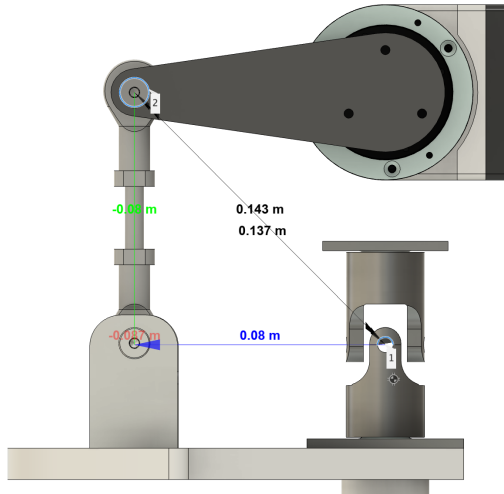
$$\begin{aligned} \mathbf{r}_{B_i} &= \mathbf{r}_{A_i} + \mathbf{R}_y(\theta_i) (\mathbf{r}_{B_i} - \mathbf{r}_{A_i}), \\ \mathbf{r}_{C_i} &= \mathbf{x}_{rot} \mathbf{r}_{C_i}, \\ \mathbf{r}_{A_i} &= \mathbf{r}_{A_i}, \end{aligned}$$

$$\mathbf{x}_{rot} = \mathbf{R}_y(q_{ankle}^{pitch}) \mathbf{R}_x(q_{ankle}^{roll}),$$

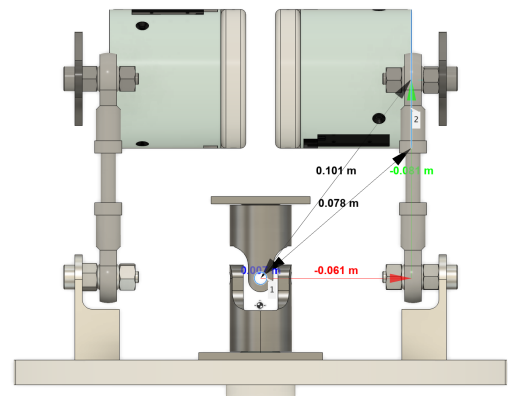
$$\mathbf{R}_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}, \quad \mathbf{R}_y(\phi) = \begin{bmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{bmatrix}.$$

For this quadruped the dimensions are:

$${}^0\mathbf{r}_{A_i} = \begin{bmatrix} 0 \\ 0.061 \\ 0.081 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ -0.061 \\ 0.081 \end{bmatrix} \quad {}^0\mathbf{r}_{B_i} = \begin{bmatrix} 0.08 \\ 0.061 \\ 0.081 \end{bmatrix}, \quad \begin{bmatrix} 0.08 \\ -0.061 \\ 0.081 \end{bmatrix} \quad {}^0\mathbf{r}_{C_i} = \begin{bmatrix} 0.08 \\ 0.061 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 0.08 \\ -0.061 \\ 0 \end{bmatrix}$$



(a) x-z plane



(b) y-z plane

Figure 3-7: Spine dimensions in x-z plane and y-z plane

3.3.2 Forward Kinematics

The forward kinematics are solved using the Newton-Raphson method estimating the error with the inverse kinematics:

$$\mathbf{x}_c^{k+1} = \mathbf{x}_c^k - (\mathbf{J}_c(\mathbf{x}_c^k))^{-1} (\mathbf{f}_{ik}(\mathbf{x}_c^k) - \boldsymbol{\theta}_{\text{ref}}), \quad (3-1)$$

$$\mathbf{J}_c = \mathbf{JG} \in \mathbb{R}^{2 \times 2}, \quad (3-2)$$

$$\mathbf{G} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \cos \alpha & 0 \\ 0 & 1 \\ -\sin \alpha & 0 \end{bmatrix}^T \in \mathbb{R}^{6 \times 2}, \quad (3-3)$$

$$\mathbf{J} = \mathbf{J}_\theta^{-1} \mathbf{J}_x \in \mathbb{R}^{2 \times 6}, \quad (3-4)$$

$$\mathbf{J}_x = \begin{bmatrix} \mathbf{r}_{\text{rod}_1}^\top (\mathbf{r}_{C_1} \times \mathbf{r}_{\text{rod}_1})^\top \\ \mathbf{r}_{\text{rod}_2}^\top (\mathbf{r}_{C_2} \times \mathbf{r}_{\text{rod}_2})^\top \end{bmatrix} \in \mathbb{R}^{2 \times 6}, \quad (3-5)$$

$$\mathbf{J}_\theta = \begin{bmatrix} \mathbf{s}_{11} \cdot (\mathbf{r}_{\text{bar}_1} \times \mathbf{r}_{\text{rod}_1}) & 0 \\ 0 & \mathbf{s}_{21} \cdot (\mathbf{r}_{\text{bar}_2} \times \mathbf{r}_{\text{rod}_2}) \end{bmatrix} \in \mathbb{R}^{2 \times 2}, \quad (3-6)$$

$$\mathbf{r}_{\text{bar}_i} = \mathbf{r}_{B_i} - \mathbf{r}_{A_i}, \quad \mathbf{r}_{\text{rod}_i} = \mathbf{r}_{C_i} - \mathbf{r}_{B_i}, \quad (3-7)$$

$$\mathbf{s}_{11} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{s}_{21} = \begin{bmatrix} 0 & -1 & 0 \end{bmatrix}. \quad (3-8)$$

Once the inverse and forward kinematics were derived, the motion strategies for gait generation and balance control could be formulated. These are presented in the following two sections.

3.4 Gait: trot equations

Trot Parameters

The foot trajectory for each leg is generated over a periodic stride using the following configurable parameters:

- T_{stride} : stride period [s]
- B : duty factor (stance ratio), $B \in (0, 1)$
- ϕ_i : phase offset for leg $i \in \{0, 1, 2, 3\}$
- L : stride length [m]
- h : apex height of the swing trajectory [m]
- x_0 : lateral position of the foot [m]
- z_0 : constant foot height during stance [m]

Normalized Time and Phase Offset

For each leg i , the current time t is normalized over the stride period and shifted by a leg-specific phase:

$$\bar{t}_i = \text{mod} \left(\frac{t}{T_{\text{stride}}} + \phi_i, 1 \right) \cdot T_{\text{stride}}$$

Let $T_{\text{stance}} = B \cdot T_{\text{stride}}$. The motion is separated into:

$$\text{Stance: } \bar{t}_i < T_{\text{stance}}, \quad \text{Swing: } \bar{t}_i \geq T_{\text{stance}}$$

Trajectory Equations

The x-coordinate is constant per leg:

$$x_i = \begin{cases} x_0 & \text{if leg } i \in \{0, 3\} \\ -x_0 & \text{if leg } i \in \{1, 2\} \end{cases}$$

A smooth quintic interpolation function is used:

$$\text{smooth}(u) = 10u^3 - 15u^4 + 6u^5, \quad u \in [0, 1]$$

Stance Phase ($\bar{t}_i < T_{\text{stance}}$): Let $u = \frac{\bar{t}_i}{T_{\text{stance}}}$. Then:

$$y_i = \begin{cases} + \left(\frac{L}{2} - L \cdot \text{smooth}(u) \right) & \text{if leg } i \in \{0, 1\} \\ - \left(\frac{L}{2} - L \cdot \text{smooth}(u) \right) & \text{if leg } i \in \{2, 3\} \end{cases}, \quad z_i = z_0$$

Swing Phase ($\bar{t}_i \geq T_{\text{stance}}$): Let $u = \frac{\bar{t}_i - T_{\text{stance}}}{T_{\text{stride}} - T_{\text{stance}}}$. Then:

$$y_i = \begin{cases} + \left(-\frac{L}{2} + L \cdot \text{smooth}(u) \right) & \text{if leg } i \in \{0, 1\} \\ - \left(-\frac{L}{2} + L \cdot \text{smooth}(u) \right) & \text{if leg } i \in \{2, 3\} \end{cases}$$

$$z_i = z_0 + \frac{h}{1.875} \cdot (30u^2 - 60u^3 + 30u^4)$$

This formulation ensures a continuous trajectory for the foot in both stance and swing phases with smooth velocity and acceleration transitions.

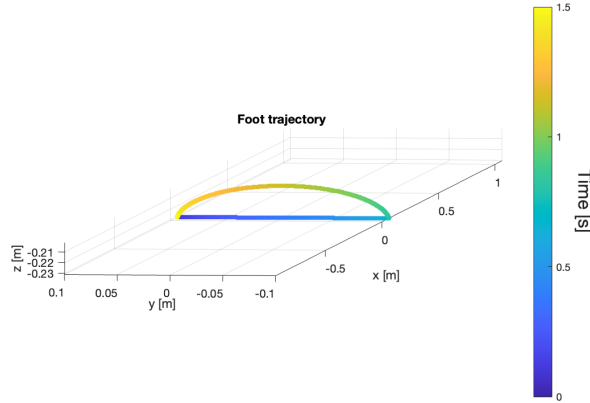


Figure 3-8: Foot trajectory over time (axis have same scale)

3.5 Balance on an inclined surface

The goal is to get a part of the body, either front or rear, to remain horizontal on an inclined plane; for this we have to give the spine a pitch angle equal to the angle of the inclined plane. Depending on which part of the body we want to keep horizontal, the angle of the inclined plane is calculated in different ways as I will show in the next paragraphs.

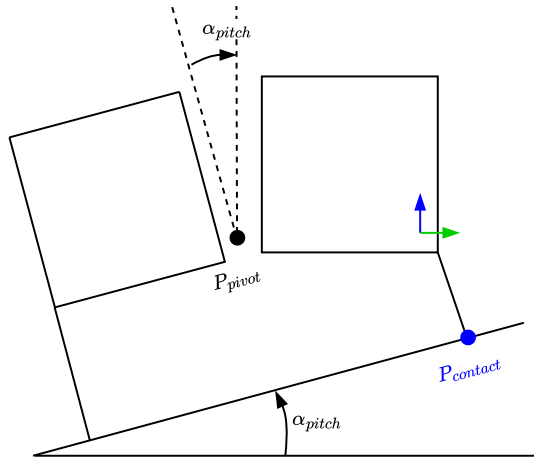


Figure 3-9: Diagram of the quadruped on an inclined plane

3.5.1 Rear body balance

For the rear body, the desired pitch angle for the spine is the angle of the IMU, $\alpha_{spine} = \alpha_{imu}$ as shown by the figure below:

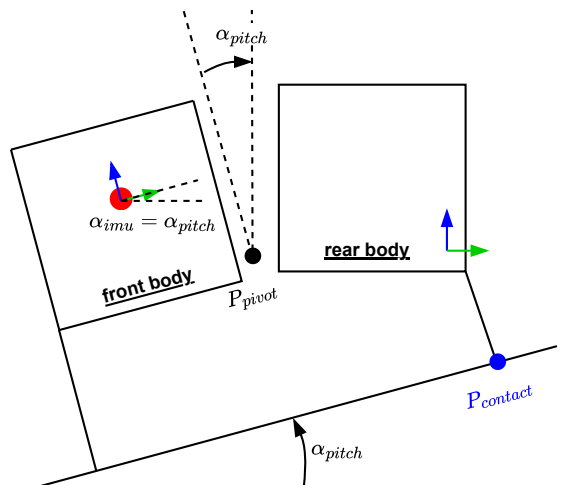


Figure 3-10: Rear body balance

3.5.2 Front body balance

However, in the case where the front body should be kept horizontal, the IMU angle will not be the angle of the inclined plane, therefore, a way through this is $\alpha_{spine+} = \alpha_{imu}$, however a gain needs to be added as otherwise the quadruped overshoots and becomes extremely unstable: $\alpha_{spine+} = k p_{spine} \cdot \alpha_{imu}$ (in my case $k p_{spine} = 0.001$)

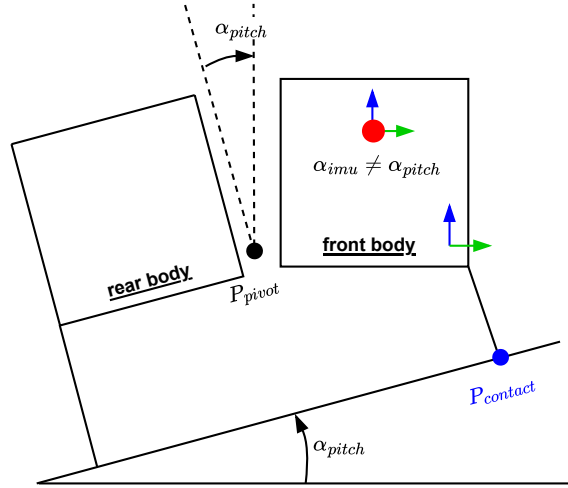


Figure 3-11: Front body balance

3.5.3 Legs

The objective is to maintain the same contact point on the ground plane while the spine rotates. This is done by expressing the original foot position p_{foot}^O (in the original leg frame O) with respect to the new rotated leg frame O' , which is centered at the same pivot point. The point rotates by an angle α_{pitch} around the x-axis, and then is translated into the leg's coordinate frame by subtracting the offset between the pivot and the leg base frame. The transformation is given by:

$$\underline{p}_{foot}^{O'} = \underline{R}_x(|\alpha_{pitch}|) \cdot \underline{p}_{foot}^O - \underline{O}$$

where the 3D rotation matrix $R_x(\alpha)$ around the x-axis is:

$$\underline{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) \\ 0 & \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

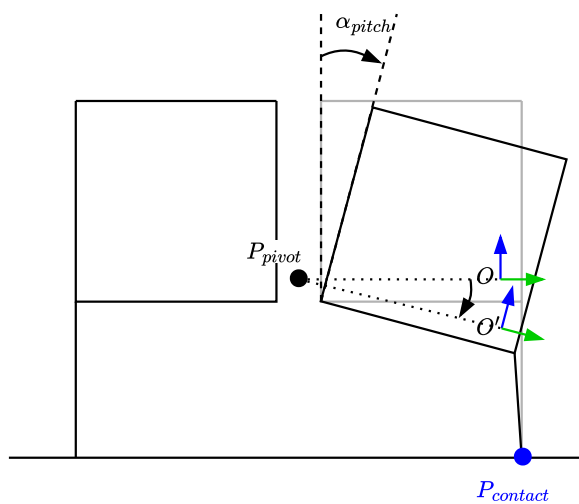


Figure 3-12: Foot position on inclined plane

4 Software Design

4.1 Introduction

The software is designed in a modular way using ROS2, allowing the physical robot to be replaced by a MuJoCo simulation without altering the control logic. Two categories of nodes are defined: **interface nodes** and **control nodes**.

The interface nodes are responsible for translating control commands to the hardware or the simulator. Specifically, `leg_command_node` communicates with the real quadruped via USB, while `mujoco_node` executes the equivalent commands in the MuJoCo simulation.

Control nodes generate the desired motion and send actuator positions to one of the two interface nodes. In this thesis, three control nodes were implemented: `hardware_check.cpp`, `trot_node.cpp`, and `spine_pitch_inclined_plane_node.cpp`. These are described in detail in Section 4.3.

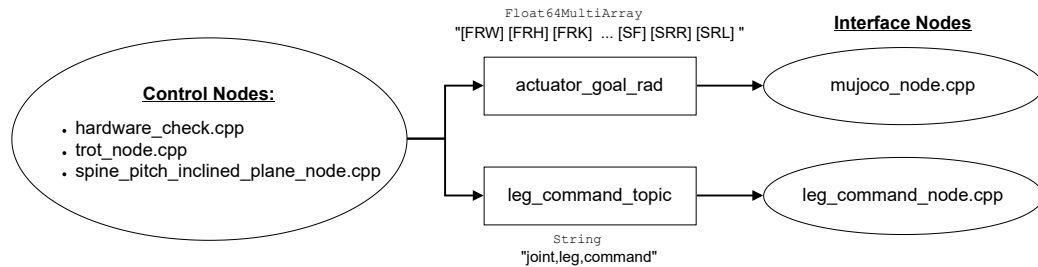


Figure 4-1: ROS2 nodes graph

4.2 Joints Notation

Each time an array refers to the four legs, the order of the legs is: front right (FR), front left (FL), rear right (RR), and rear left (RL). If the array also contains the joints of each leg, the order of the joints is: waist (W), hip (H), and knee (K). The spine values, when present, are always placed last, with the joint order: front (F), rear right (RR), rear left (RL).

Table 4-1 summarizes the arrangement: each row corresponds to a joint type, and each column corresponds to a leg or spine element.

	FR (col=0)	FL (=1)	RR (=2)	RL (=3)	S (=4)
W (line=0)	FR-W	FL-W	RR-W	RL-W	S-F
H (=1)	FR-H	FL-H	RR-H	RL-H	S-RR
K (=2)	FR-K	FL-K	RR-K	RL-K	S-RL

Table 4-1: Joint and leg order in arrays.

The next section will explain the code of each node, in the following order:

- `leg_command_node.cpp`
- `mujoco_node.cpp`
- `hardware_check.cpp`
- `trot_node.cpp`
- `spine_pitch_inclined_plane_node.cpp`

4.3 ROS 2 architecture

4.3.1 `leg_command_node.cpp`

This node receives commands in 'human language' through the `leg_command_topic` and translates them into USB messages sent to the STM32s.

The topic to which commands should be published is `leg_command_topic`, using the format `"leg,joint,command"` (`std_msgs/msg/String`). For clarity, leg and joint numbering starts at 1, as it is more intuitive to refer to the first leg rather than leg zero. For example:

1, 1 = front right waist joint, 2, 3 = front left knee joint, 4, 2 = rear right hip joint.

The `command` field can take one of the following values:

- `enable`
- `disable`
- `request`
- `set_zero_position`
- `mit,kp,kd,pos,vel,tau`

Examples:

- **"1,2,enable"** enables the hip motor of the front right leg.
- **"3,1,set_zero_position"** sets the current position as zero for the waist motor of the rear right leg.
- **"4,1,request"** requests the current position of the waist actuator of the rear left leg.
- **"2,3,mit,6,0.6,0,0"** sets the knee motor of the front left leg to MIT control with $k_p = 6$, $k_d = 0.6$, position = 0 rad, angular velocity = 0 rad/s, and torque = 0 Nm.

This structure allows new control nodes to be added easily to send commands. For the MuJoCo simulation, the format of the message published to `actuator_goal_rad` is even simpler (cf. next section).

Sending individual commands from the terminal

One of the main reasons for designing this node to send only a single command at a time rather than sending the positions of all 15 actuators at once, as done in the `mujoco_node`, is to enable direct control of any specific motor from the terminal. This approach allows testing or debugging individual actuators without modifying the control code.

For example, the following command sends an MIT control instruction to the hip motor of the front left leg:

```
ros2 topic pub -1 /leg_command_topic std_msgs/msg/String \
"data: '2,2,mit,4.0,0.4,0,0,0'"
```

Code structure and message flow

The `leg_command_node.cpp` is designed as a modular pipeline: each function has a single role. When a command message is published to `leg_command_topic`, it goes through four steps (Figure 4-2):

1. **Message reception:** The ROS 2 subscriber callback calls `handle_cmd()`, which parses the incoming "leg,joint,command" string and extracts numeric values and the command type.
2. **Payload preparation:** One of three functions builds the 8-byte CAN payload: `prepare_payload_cmd()` (enable/disable/zero position), `prepare_payload_mit()` (MIT control), or `prepare_payload_request()` (register read).
3. **USB frame assembly:** The CAN payload is passed to `build_frame()` to add CAN ID/bus, then `build_usb_frame()` wraps it into the 14-byte USB packet.
4. **USB transmission:** `send_usb()` writes the packet to the correct serial port (`/dev/ttyACM0` or `/dev/ttyACM1`) and optionally waits for a reply.

This modular design means adding a new command type only requires creating a new payload preparation function without changing the rest of the process. A collapsed version of the code is presented in the Appendix A.1.

Execution flow diagram:

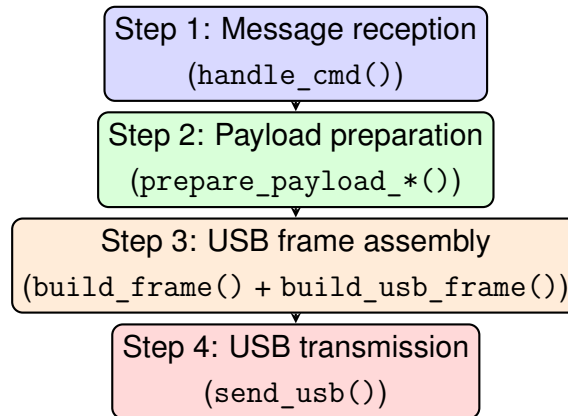


Figure 4-2: Function call sequence in `leg_command_node.cpp`

4.3.2 mujoco_node.cpp

This node runs the MuJoCo physics simulation when launched, where each joint is controlled to the requested position by the topic `actuator_goal_rad`.

It subscribes to the topic `actuator_goal_rad` (**type:** `std_msgs/msg/Float64MultiArray`), which carries the desired joint positions for the four legs and the spine, in the same leg/joint ordering convention used throughout the project. The received positions are stored in the array `actuator_goal_rad[joint][leg/spine]`.

The node can operate in two modes: **torque control** or **position control**. - In torque control mode, it computes joint torques using a custom PD controller based on measured positions and velocities from the simulation. - In position control mode (used for most of this thesis), the desired joint positions from `actuator_goal_rad` are sent directly to the simulation actuators.

I chose to keep the torque control functions in the codebase (`get_cur_actuator_pos()`, `get_cur_actuator_vel()`, `computePDtorque_fixed()`) so that switching to torque mode in the future only requires uncommenting a few lines in the `controller()` function, which orchestrates all major control steps.

The node also publishes simulated IMU measurements to the topic `imu_data` (**type:** `sensor_msgs/msg/Imu`), which are primarily used for spine control during balance experiments.

Code structure and message flow for position control:

1. **Message reception:** Subscription callback to `actuator_goal_rad` updates the `actuator_goal_rad` array.
2. **Command sending:** The desired positions are written directly to `d->ctrl[]` using `send_cmd_pos()`.

Code structure and message flow for torque control:

1. **Message reception:** Subscription callback to `actuator_goal_rad` updates the `actuator_goal_rad` array.
2. **State acquisition:** `get_cur_actuator_pos()` and `get_cur_actuator_vel()` read the simulated joint states from MuJoCo sensors.
3. **Control computation:** `computePDtorque_fixed()` calculates the torque for each joint from PD gains and position/velocity errors.
4. **Command sending:** The torques are written to `d->ctrl[]` using `send_cmd_torque()`.

Again, a collapsed structure of the code is presented in the Appendix A.1.

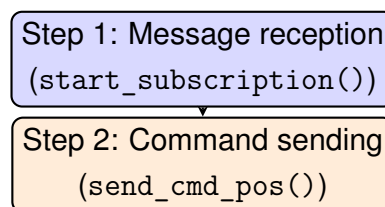
Execution flow diagram for position control:

Figure 4-3: Function call sequence for position control in `mujoco_node.cpp`

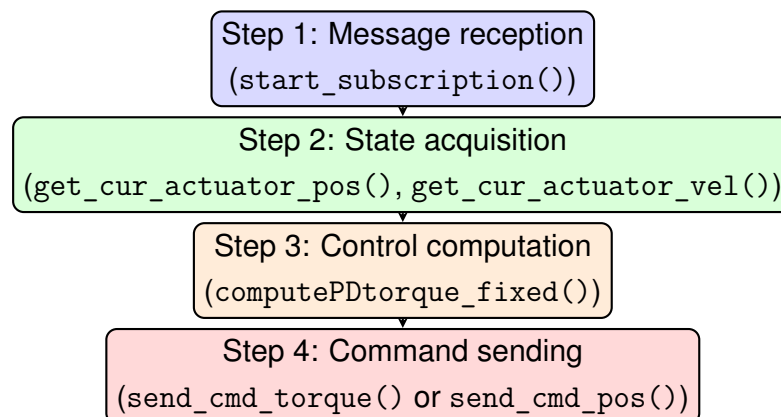
Execution flow diagram for torque control:

Figure 4-4: Function call sequence in `mujoco_node.cpp`

4.3.3 hardware_check.cpp

This node provides a simple routine for setting all the actuators' mechanical position to zero.

On startup it zeroes all joints and drives them to a no-torque MIT state, then iterates leg-by-leg in the typical order FR, FL, RR, RL, S (legs 1 to 5) every 5 s to zero the legs joints again (so the user can choose the mechanical zero) and apply an MIT hold at 0 rad (to keep the chosen position). All commands are published to `leg_command_topic` (`std_msgs/msg/String`) using the same "leg,joint,command" format as the other nodes.

Default startup behavior (on node creation):

1. **Enable all motors without torque** (`do_startup_sequence(6)`): for each leg 1...5 and joint 1...3: disable $\rightarrow$ enable $\rightarrow$ set_zero_position $\rightarrow$ mit,0,0,0,0,0 (no torque).
2. **Per-leg initialization** (every 5 s in order FR, FL, RR, RL, S): set_zero_position $\rightarrow$ mit,1.0,0.1,0,0,0 for joints 1,2,3 of that leg.

Typical usage. Launch the node and use the 5 s window to manually place the **front right** leg at the desired zero (cf. IK leg). At $t = 5$ s the node records zero and applies a gentle hold; the next 5 s window is for the **front left**, then **rear right**, **rear left**, and finally the **spine**.

Available debug helpers (uncomment to use):

- `send_mit_cst_test()`: periodically sends mit,5,0.5,0,0,0 to all joints (attachable to a 50 ms wall timer).
- `send_position_requests()`: requests the position of all 15 joints.
- `send_disable_all()`: disables every actuator.

Again, a collapsed structure of the code is presented in the Appendix A.1

Execution flow diagram of the startup sequence:

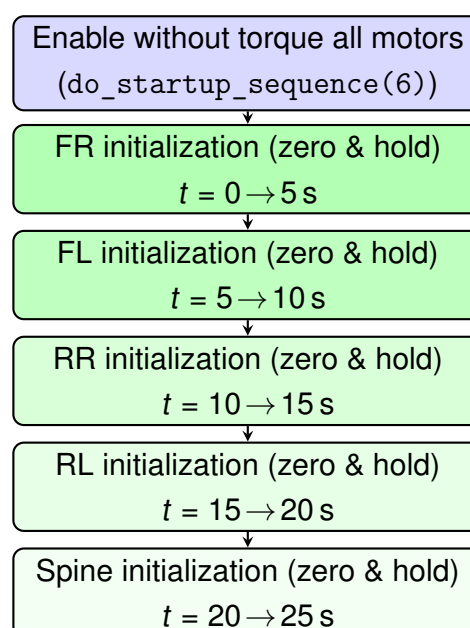


Figure 4-5: Function call sequence in *hardware_check.cpp*

4.3.4 trot_node.cpp

This control node computes the trot equations, from which it calculates the joints' position with IK and finally publishes the commands to an interface node.

For clarity, the implementation is organized into three parts: trot equation, IK, and the controller (which publishes).

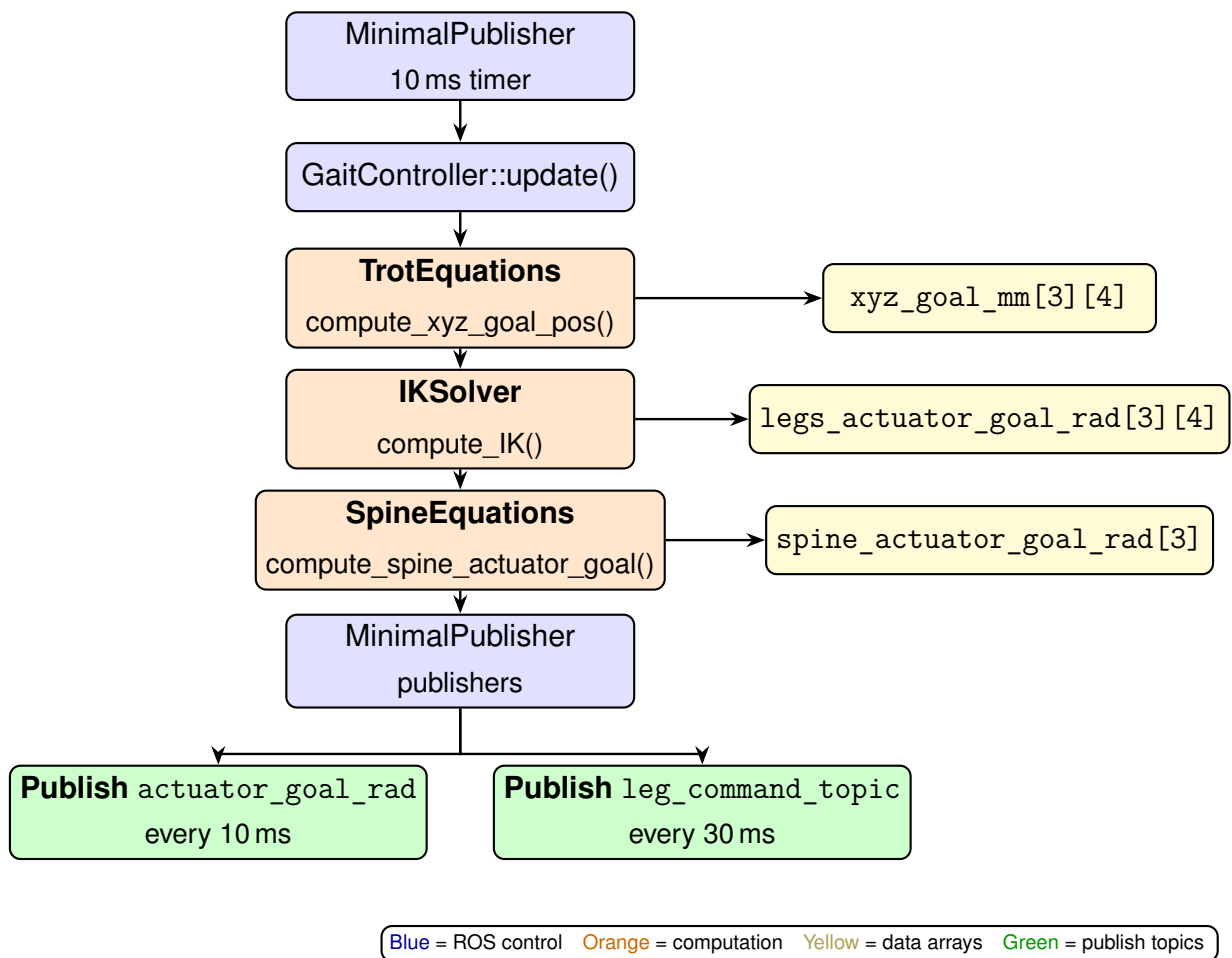
Collapsed view of the file (classes and entry point):

```

1 double xyz_goal_mm[3][4] = {0};
2 double legs_actuator_goal_rad[3][4] = {0};
3 double spine_actuator_goal_rad[3] = {0};
4 class TrotEquations { /* compute_xyz_goal_pos(), stance(), swing(), ...
   */ };
5 class IKSolver { /* compute_IK(), link lengths d1,a2,a3 */ };
6 class SpineEquations{ /* compute_spine_actuator_goal() -> zeros */ };
7 class GaitController{ /* update(): trot -> IK -> spine */ };
8 class MinimalPublisher : public rclcpp::Node { /* timers + publishers */
   };
9 int main(int argc, char* argv[]) { /* spin MinimalPublisher */ }

```

Step-by-step process of trot_node.cpp:

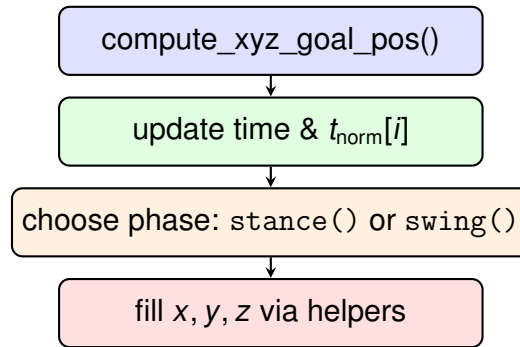


1) Trot equations

TrotEquations computes for each leg the coordinates of the end-effector desired position.

It keeps all the gait parameters in one place (stride period, duty factor, swing height, stride length, leg phase offsets, base height, ...). At each control step, one call to `compute_xyz_goal_pos()` refreshes the four foot targets. Internally it decides whether a leg is in *stance* or *swing* and then fills x , y , z using the small helper functions. A collapsed version of the code is presented in the Appendix A.1.

Call flow inside `compute_xyz_goal_pos()`:



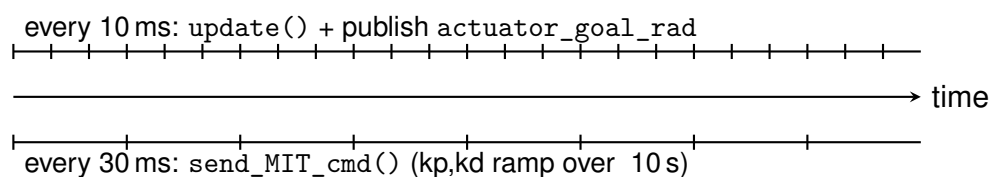
2) Legs & spine actuator positions (IK)

`IKSolver::compute_IK()` turns the foot targets into joint angles [waist, hip, knee] for each leg. `SpineEquations` is a small class kept for future experiments; for now its function simply sets the three spine goals to zero. A collapsed version of the code is presented in the Appendix A.1.

3) Controller and publishers

`GaitController::update()` calls the three steps in order: trot equations $\rightarrow$ IK $\rightarrow$ spine. In `MinimalPublisher`, this update runs every **10 ms** and, in the same timer, publishes the **15 actuator goal positions** to `actuator_goal_rad` so you can see the whole set of actuator positions in one line at each time step. A second timer publishes MIT commands to `leg_command_topic` every **30 ms**. The gains use a **ramp** that goes from 0% to 100% over **10 s** to keep bring-up smooth. A collapsed version of the code is presented in the Appendix A.1

Timing overview:



4.3.5 spine_pitch_inclined_plane_node.cpp

This control node keeps either the front or the rear of the robot horizontal while standing on an inclined plane.

A single flag routes almost all computations:

```

int incline_front_or_rear = 1;
{
  0 : keep front horizontal
  1 : keep rear horizontal
}
  
```

The controller uses the IMU pitch and the current spine positions to compute spine and leg targets that satisfy the chosen constraint.

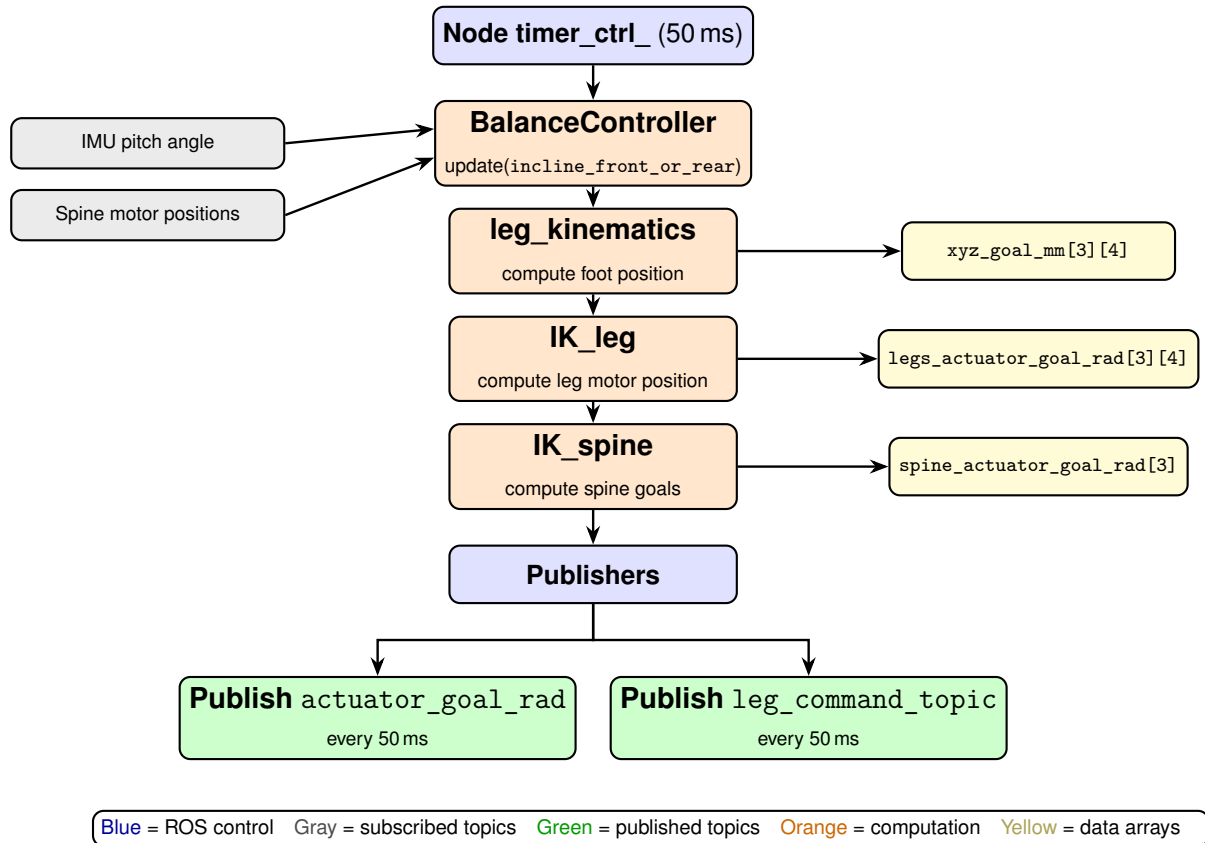
Collapsed view of the file (classes and entry point):

```

1 double xyz_goal_mm[3][4] = {0};           // foot xyz goals (FR, FL,
   RR, RL)
2 double legs_actuator_goal_rad[3][4] = {0}; // [waist, hip, knee] per
   leg
3 double spine_actuator_goal_rad[3] = {0};   // [front, rear-right, rear
   -left]
4
5 class leg_kinematics { /* inclined foot pose from IMU & offsets
   */ };
6 class IK_leg { /* leg IK -> legs_actuator_goal_rad
   */ };
7 class IK_spine { /* spine IK -> spine_actuator_goal_rad
   */ };
8 class FK_spine { /* FK from spine angles (consistency /
   estimation) */ };
9 class BalanceController{ /* orchestrates front/rear balance logic
   */ };
10
11 class spine_pitch_inclined_plane_node : public rclcpp::Node { /* I/O +
   timers */ };
12 int main(int argc, char** argv) { /* spin node */ }

```

Step-by-Step Process of spine_pitch_inclined_plane_node.cpp



1) Kinematics

Exactly the same data layout as in the trot node is used:

`xyz_goal_mm[3][4]`, `legs_actuator_goal_rad[3][4]`, `spine_actuator_goal_rad[3]`.

The `leg_kinematics` class calculates new foot position for either the front or the rear, depending on `incline_front_or_rear`, to keep the foot fixed on the inclined plane. Then, the `IK_leg` class computes the joints' positions for the affected pair of legs, and sets the other pair to zero. Finally, the `IK_spine` class maps the requested body pitch into the two rear spine actuators (front spine stays at zero). Also, a `FK_spine` routine is available, as it was previously used to get the current pitch angle of the spine from the current spine motor position.

2) Control

`BalanceController::update()` runs every **50 ms**. With `incline_front_or_rear=1` (default), it keeps the **rear** horizontal by using the IMU pitch as the spine pitch request; the rear legs follow the inclined plane while the front legs stay at zero. With `incline_front_or_rear=0`, the **front** is kept horizontal: the node first estimates the pose via `FK_spine`, then slowly integrates the IMU pitch (k_p -like term) to shape the target and updates foot/leg IK accordingly (cf Chapter 4, Section 4 on control techniques used for balance)

3) Subscribers and publishers

Subscribers

- `/euler_only` or `imu_data`: IMU pitch used for balancing.
- `joint_pos_topic`: spine motor positions, needed for FK_spine.

Publishers

- `actuator_goal_rad` (`Float64MultiArray`, every **50 ms**): one single line, with the 12 leg goals followed by the 3 spine goals.
- `leg_command_topic` (`String`, every **50 ms**): MIT commands for all 15 actuators.

Note. In the control timer, the node first requests the three spine positions, updates the controller, prints the full goal vector for logging, and then publishes to both topics.

4.4 MuJoCo Simulation

The MuJoCo simulation was configured using a URDF file exported from the Fusion 360 model of the quadruped. Since ball joints are not supported in the URDF export, all ball joints in the Fusion model had to be replaced with hinge joints prior to export. After generating the URDF, these hinge joints were manually converted back to ball joints. However, MuJoCo does not support multiple ball joints on a single body. To work around this limitation, the spine links were split into two separate bodies and reconnected using the `weld` constraint. This approach is not entirely rigid, as the weld behaves more like a 1D spring-damper system attempting to hold the parts together, rather than enforcing a perfectly fixed connection. Nevertheless, this level of accuracy is sufficient for the simulation, whose primary purpose is to verify that the control commands do not risk damaging the physical prototype. An alternative approach that was also tested was to remove the spine links entirely and instead drive the motion using only a universal joint. This method worked particularly well as this solution eliminates the issue of the spine link not being perfectly rigid.

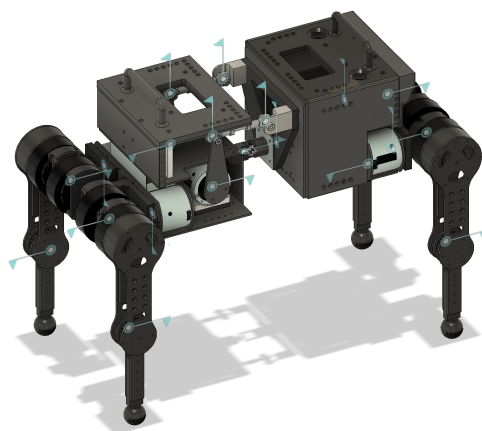


Figure 4-6: Rendering of the quadruped with the 19 hinge joints for URDF export

The simulation was integrated into ROS 2 as a node that subscribes to the topic `actuator_goal_rad`. This topic publishes a 15×1 array containing the desired actuator positions in radians for all joints. The ordering of the array is as follows:

[FR waist, FR hip, FR knee, FL waist, FL hip, FL knee, RR waist, RR hip, RR knee, RL waist, RL hip, RL knee, Front spine, RR spine, RL spine]

4.4.1 Control

The MuJoCo environment's integrated control system was used for this simulation. Its parameters can be adjusted in the following section of the code (line 25):

```
1 // Controller-related variables
2 float_t ctrl_update_freq = 100;
3 mjtNum last_update = 0;
4 mjtNum ctrl;
```

If proportional-derivative (PD) control is preferred, the actuators in the XML file must be changed from `position` to `motor`.

4.4.2 XML Models

Two XML models are available:

- **model.xml** Includes the full spine model and the `weld` constraint.
- **quadruped_UJ_activated.xml** Uses only a universal joint for the spine, which directly drives the motion.

Switching between the two is straightforward: modify the first line after the includes (line 23) to select the desired file:

```
1 char filename[] = "quadruped_UJ_activated.xml";
```

4.4.3 Sensor IDs

To simplify model switching and improve code readability, sensor IDs are managed through dedicated `.cpp` and `.hpp` files. This allows sensors to be referenced by name rather than by index. For example:

```
1 cur_actuator_pos[1][1] = d->sensordata[m->sensor_adr\[id_FL_femur_s_pos
  ]];
```

Note that, due to the initial implementation, the `waisthipknee` joints are internally referred to as `hipfemurtibia`.

Adding new sensors is straightforward:

1. Declare them in `sensor_ids.hpp` (as `extern int`).
2. Define them in `sensor_ids.cpp` and add them to the `init_sensor_ids()` function.

4.4.4 Inclined Plane

A dedicated class was implemented to simulate an inclined plane. The parameters can be modified at the beginning of the code (line 30):

```
1 // Inclined plane parameters
2 float T_incline = 5; // Time to reach max_degree
3 float max_degree = 0; // Starts at 0°, reaches max_degree in T_incline
   seconds
```

5 Testing and Results

5.1 Testing Procedure and Recommendations

This section describes the testing procedure for the quadruped robot and provides practical recommendations based on the work carried out.

5.1.1 MIT Position Limits

To ensure safety in all commands requested by the user or the code, a safety check for the MIT command limits is implemented in `prepare_payload_mit()`. These values can be modified within the function:

```
1 float q_limit_min = -M_PI;
2 float q_limit_max = M_PI;
3
4 if (can_bus == 5) {
5     q_limit_min = -40 * DEG2RAD;
6     q_limit_max = 40 * DEG2RAD;
7 } else {
8     if (can_id==1) {q_limit_min = -30 * DEG2RAD; q_limit_max = 30
9         * DEG2RAD; }
10    else if (can_id==2) {q_limit_min = -60 * DEG2RAD; q_limit_max = 60
11        * DEG2RAD; }
12    else if (can_id==3) {q_limit_min = -135 * DEG2RAD; q_limit_max = 135
13        * DEG2RAD; }
14 }
```

If the values exceed the hardware limits, the command is ignored and an error is logged in the terminal.

5.1.2 Communication Setup

It is essential to connect devices in the following order to ensure correct `/dev/ttyACM` assignments:

1. Front STM32 becomes `tttyACM0`
2. Rear STM32 becomes `tttyACM1`
3. IMU automatically detected

5.1.3 Useful Commands

Before running any ROS 2 nodes, ensure you are in the workspace and source the environment:

```
1 source install/setup.bash
```

I listed some commonly used commands:

- Start IMU:

```
1 ros2 launch yesense_std_ros2 yesense_node.launch.py
```

- Run the leg command node:

```
1 ros2 run leg_command leg_command_node
```

- List connected USB devices:

```
1 ls -l /dev/ttyACM*
```

- Send a leg command:

```
1 ros2 topic pub -1 /leg_command_topic std_msgs/msg/String \
2 "data:␣'2,2,mit,4.0,0.4,0,0,0'"
```

Replace `mit` with `enable`, `disable`, `request`, or `set_zero_position` as needed.

- List all ROS topics:

```
1 ros2 topic list
```

- Print euler_only topic data:

```
1 ros2 topic echo /euler_only
```

5.1.4 Final Recommendations

Before testing new motors or scripts:

- Always check motor position, speed and torque limits with the app provided by Damiao, as the position, velocity or torque value sent with MIT is not an absolute value but a percentage of the maximum value (cf Chapter 3, Section 3 Electronics Hardware)
- Verify K_p and K_v limits (same reason)
- Ensure these values match those used in the MIT commands.
- Keep all motors updated with the latest firmware, or at least ensure they are running the same firmware version.

5.2 Results

5.2.1 Hardware Analysis

Electronics Integration

The chosen hybrid architecture, combining a Raspberry Pi 5 with STM32 microcontrollers, proved highly effective during testing. The modular design allowed the Raspberry Pi to act purely as a high-level controller, while the STM32 boards handled the low-level CAN communication in real time. This not only improved reliability but also provided flexibility: the Raspberry Pi can be easily removed or replaced within minutes, and the system remains compatible with more powerful computers if required in the future. Overall, this configuration demonstrated robustness, portability, and straightforward maintainability.

Mass Distribution

In addition to performance, the mass distribution of the quadruped was analyzed to identify how different subsystems contribute to the total weight. Table 5-1 presents a breakdown of the primary components and their respective contributions, expressed both in absolute mass and as a percentage of the total robot weight. In particular, the actuators represent the dominant fraction of the total mass (52%), which is typical for quadrupeds of this scale.

Table 5-1: Mass distribution of quadruped components.

Subsystem / Part	Material	Units	Mass / unit [kg]	Total [kg]
Actuators				
DM6006	—	9	0.35	3.15
DM4010	—	6	0.30	1.80
Subtotal Actuators				4.95
Spine				
Front actuator support	Tough PLA	1	0.055	0.055
Rear actuator support	Tough PLA	2	0.055	0.110
Spine support	Tough PLA	1	0.165	0.165
Link (w/ ball joints)	Steel	2	0.050	0.100
Bar	CFRP	2	0.012	0.024
Universal joint	Steel	1	0.220	0.220
Subtotal Spine				0.674
Rear body structure				
Rear frame	CFRP	1	0.365	0.365
Cover plates	Tough PLA	4	0.096	0.384
Structural supports	Tough PLA	1	0.172	0.172
Column	Aluminium	2	0.005	0.010
Electronics support	Tough PLA	1	0.004	0.004
Subtotal Rear body				0.935
Front body structure				
Front frame	CFRP	1	0.401	0.401
Cover plates	Tough PLA	1	0.469	0.469
Supports	Tough PLA	1	0.149	0.149
Column	Aluminium	4	0.005	0.020
Electronics support	Tough PLA	1	0.028	0.028
Subtotal Front body				1.039
Electronics				
Raspberry Pi 5	—	1	0.064	0.064
STM32 Board	—	2	0.025	0.050
Robomaster PDU	—	2	0.025	0.050
Battery	—	1	0.650	0.650
Wires	—	—	—	0.05
Subtotal Electronics				0.864
Leg				
Frame	CFRP	1	0.100	0.100
Waist to hip support	Tough PLA	1	0.073	0.073
Hip to knee support	Tough PLA	1	0.050	0.050
Foot structure	Tough PLA	1	0.025	0.025
Gears for toothed belt	Tough PLA	1	0.031	0.031
Subtotal Leg				0.279
Total Robot Mass				9.5

Figure 5-1 complements the table with a pie chart representation, where the dominant contribution of the actuators becomes evident. Such an analysis is useful not only to identify optimization opportunities (e.g., reducing actuator weight or redesigning the frame to shift the distribution), but also to benchmark the robot against other quadrupeds in terms of actuator-to-weight ratio.

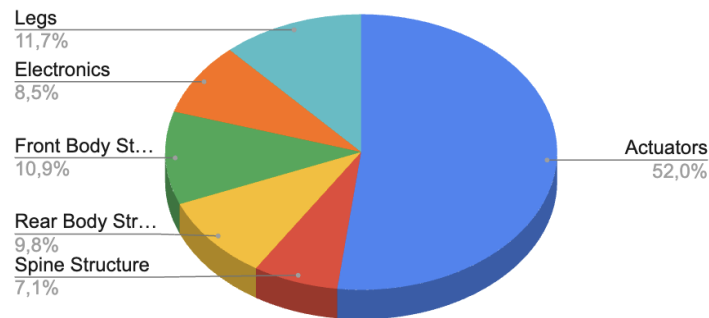


Figure 5-1: Mass distribution of the quadruped.

5.2.2 Test cases choice

The main objective of this thesis was to develop a research platform for investigating active spine designs. Accordingly, the test cases were selected based on the following criteria:

- **Leg strength:** the ability to sustain the quadrupeds weight while still enabling locomotion.
- **Spine strength:** the ability to emulate a rigid chassis during trotting and to balance the quadruped with sufficient speed and stability.

One of the test cases had to demonstrate that the quadruped is capable of walking. However, many distinct gaits are defined in the literature:

- **Walk:** the least tiring gait, consisting of a four-beat sequence in which each limb executes the cycle of liftswingsupportthrust in turn. At least two or three legs are always in contact with the ground, making it very stable.
- **Amble:** a two-beat gait where the forelimb and hindlimb on the same side move together. It is smoother than the walk and is often used as an intermediate-speed gait in animals such as horses.
- **Trot:** a two-beat diagonal gait in which the forelimb and hindlimb on opposite sides move together. It is symmetrical, relatively energy-efficient, and provides good stability at moderate speeds. This is also the most common gait used in quadruped robotics.
- **Pace:** a two-beat lateral gait in which the forelimb and hindlimb on the same side move together. Compared to the trot, it is less stable laterally but allows faster oscillations.
- **Canter:** a three-beat gait that is asymmetric; the sequence involves one hindlimb, followed by the opposite hindlimb and diagonal forelimb together, and finally the remaining forelimb. It provides smoother acceleration and is often used before transitioning to gallop.

- **Transverse gallop:** a four-beat gait where the sequence starts from a hindlimb, then the opposite hindlimb, followed by the opposite forelimb, and finally the leading forelimb. It is the typical gallop seen in horses and allows high speed but with higher energy cost.
- **Rotatory gallop:** also a four-beat gait but with a different sequence, where the hindlimbs move one after the other and then the forelimbs. This gait is used by carnivores such as cheetahs and greyhounds, allowing very high speeds with the help of spinal flexion and extension.

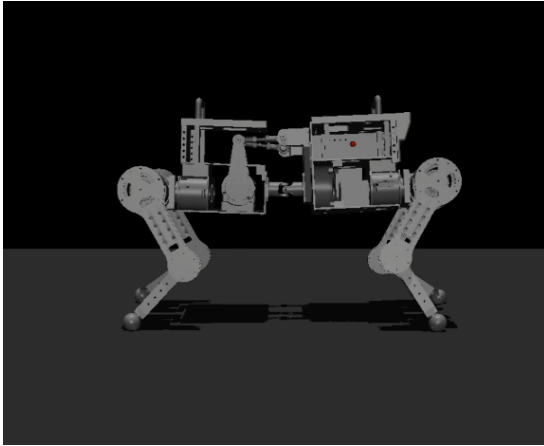
After thorough consideration, the chosen gait for this project was the trot, as it is widely adopted in quadruped robotics for its balance of speed, energy efficiency, and stability. During gait execution, the spine must be sufficiently rigid to ensure that the body behaves as a stable chassis. This rigidity is essential in the current implementation, since it allows the gait dynamics to be analyzed without additional complexity from spinal motion. At the same time, maintaining the spine as a passive rigid structure provides a valuable baseline: in future work, it will be possible to compare this “rigid body” case directly with an active spine on the same quadruped platform. Such a comparison would be particularly relevant, as biological observations show that animals exploit spinal flexion and extension to increase stride length, energy efficiency, and speed. By first validating a rigid-body gait on this quadruped, subsequent experiments with an actuated spine could highlight the benefits in terms of locomotion performance, stability on uneven terrain, and agility during dynamic maneuvers such as jumps or sprints.

Another test was designed specifically for the spine, in order to demonstrate that it is also capable of controlling its degrees of freedom. A suitable and fast-to-implement experiment was to evaluate balance on an inclined surface, by controlling only the pitch angle of the spine. This test is sufficient to show that the actuated spine can provide corrective motion that contributes to whole-body stability. Naturally, more complex extensions are possible. For instance, the control could be extended to include not only the pitch but also the roll angle of the spine, requiring a coordinated strategy to combine both degrees of freedom. Nevertheless, restricting the experiment to pitch control was adequate for the purposes of this work, as it provided a clear and measurable demonstration of spinal actuation without introducing unnecessary complexity at this stage.

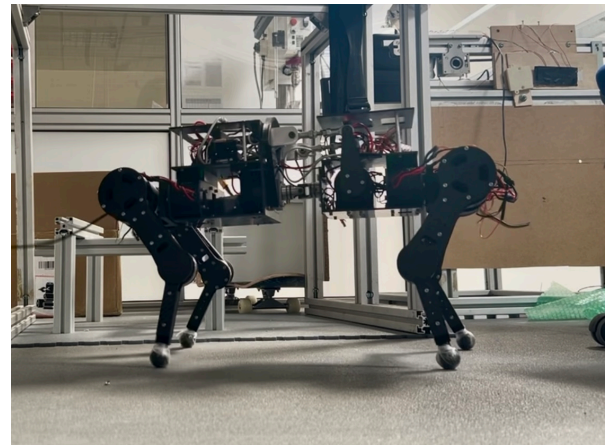
In the end, two primary tests were conducted to assess performance:

- Making the quadruped trot.
- Making the quadruped balance on an inclined plane.

All algorithms were first validated in simulation before being successfully implemented on the real robot. Since the trot relies on a hard-coded closed-loop trajectory, the achieved walking speed was approximately 4 cm/s. Figures 5-2 and 5-3 show the results obtained in simulation and on the physical platform, respectively.

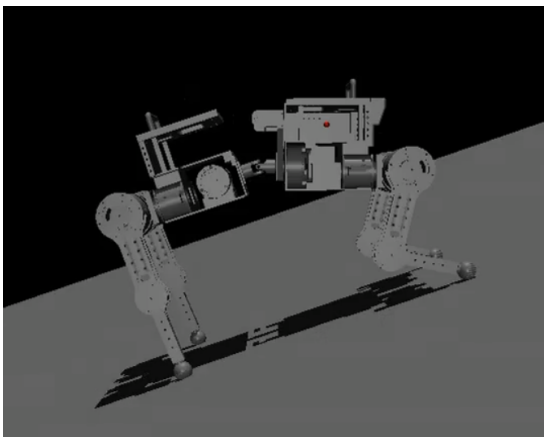


(a) Mujoco simulation

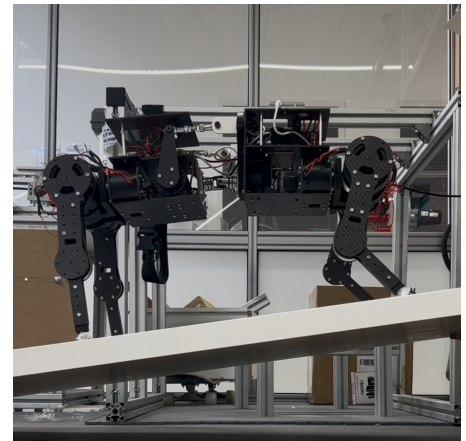


(b) Results on quadruped

Figure 5-2: Simulation and results of trot test



(a) Mujoco simulation



(b) Results on quadruped

Figure 5-3: Simulation and results of balance on inclined surface test

6 Conclusion

The primary objective of this thesis was to design and develop a quasi-direct drive quadruped robot with an actuated spine, providing a robust research platform for future investigations. This goal was successfully achieved: the robot, named ASTRA, is fully controllable, capable of walking using a trot gait, and able to balance on an inclined surface. These demonstrations validate both the mechanical robustness of the design and the reliability of the embedded electronics and software architecture.

From a technical perspective, the work encompassed the complete development cycle of a robotic platform. The hardware design integrated lightweight CFRP structures with 3D-printed supports, optimized actuator placement, and a parallel spine mechanism capable of three degrees of freedom. The electronics architecture combined a Raspberry Pi 5 with STM32 microcontrollers, striking a balance between modularity, cost, and robustness. On the software side, ROS 2 nodes were implemented for gait generation, balance control, and hardware interfacing, with all algorithms first validated in a MuJoCo simulation before being deployed to the physical prototype. The results confirm that the chosen quasi-direct drive actuation scheme provides both transparency and sufficient torque, while the active spine expands the potential for more dynamic and adaptive locomotion.

Beyond the technical outcomes, this project was also a valuable personal milestone. As a mechanical engineer, it allowed me to extend my expertise into electronics and embedded software. The laboratory environment offered the opportunity to collaborate with PhD students and gain insight into ongoing research, giving me a clearer understanding of the challenges and directions in modern robotics. This experience reinforced my motivation to pursue robotics further and to contribute at the research level during my Masters studies.

Looking forward, several research directions remain open. The current control strategies were deliberately kept simple, with a hard-coded trot gait and basic spine pitch control. More advanced approaches such as Model Predictive Control, Central Pattern Generators, or reinforcement learning could leverage the full capabilities of the active spine. Additional studies on energy efficiency and dynamic maneuvers (e.g., jumps or sprints) would provide valuable insights into the role of spinal actuation in legged locomotion. Finally, the present mechanical design provides a solid foundation that can be further optimized if the quadruped is developed into a larger-scale research project in the future.

In conclusion, the thesis achieved its goal of building a reliable and versatile spinal quadruped, establishing a solid foundation for future research. ASTRA demonstrates that active spines are not only mechanically feasible but also open new avenues for studying agility, efficiency, and adaptability in quadruped robots.

Literaturverzeichnis

- [Rai-1986] M. H. Raibert, *Legged Robots That Balance*. Cambridge, MA: MIT Press, 1986.
- [Lee-1996] K. F. Leeser, “Locomotion Experiments on a Planar Quadruped Robot with Articulated Spine”, M.S. thesis, Massachusetts Institute of Technology, 1996.
- [Rai-2008] M. Raibert, K. Blankespoor, G. Nelson, and R. Playter, “BigDog, the Rough-Terrain Quadruped Robot”, in *Proceedings of the 17th World Congress of the International Federation of Automatic Control (IFAC)*, vol. 41, Elsevier, 2008, pp. 10 822–10 825. DOI: 10.3182/20080706-5-KR-1001.01833.
- [Sem-2011] C. Semini, N. G. Tsagarakis, E. Guglielmino, M. Focchi, F. Cannella, and D. G. Caldwell, “Design of HyQ – a Hydraulically and Electrically Actuated Quadruped Robot”, *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 225, no. 6, pp. 831–849, 2011. DOI: 10.1177/0959651811402275.
- [Hut-2016] M. Hutter, C. Gehring, D. Jud, *et al.*, “ANYmal – a Highly Mobile and Dynamic Quadrupedal Robot”, in *Proceedings of the 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2016, pp. 38–44. DOI: 10.1109/IROS.2016.7758092.
- [Bel-2017] D. C. Bellicoso, F. Jenelten, P. Fankhauser, C. Gehring, J. Hwangbo, and M. Hutter, “Dynamic Locomotion and Whole-Body Control for Quadrupedal Robots”, in *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 3359–3365. DOI: 10.1109/IROS.2017.8206174.
- [Ble-2018] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, “MIT Cheetah 3: Design and Control of a Robust, Dynamic Quadruped Robot”, in *Proceedings of the 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 2245–2252. DOI: 10.1109/IROS.2018.8593885.
- [DiC-2018] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control”, in *Proceedings of the 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 1–9. DOI: 10.1109/IROS.2018.8594448.
- [Kat-2019] B. Katz, J. Di Carlo, and S. Kim, “Mini Cheetah: A Platform for Pushing the Limits of Dynamic Quadruped Control”, in *Proceedings of the 2019 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2019, pp. 6295–6301. DOI: 10.1109/ICRA.2019.8793865.

- [Kau-2019] N. Kau, A. Schultz, N. Ferrante, and P. Slade, "Stanford Doggo: An Open-Source, Quasi-Direct-Drive Quadruped", in *Proceedings of the 2019 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2019, pp. 6309–6315. DOI: 10.1109/ICRA.2019.8794436.
- [Hwa-2019] J. Hwangbo, J. Lee, A. Dosovitskiy, *et al.*, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, eaau5872, 2019. DOI: 10.1126/scirobotics.aau5872.
- [Kho-2013] M. Khoramshahi, A. Spröwitz, A. Tuleu, M. N. Ahmadabadi, and A. J. Ijspeert, "Benefits of an Active Spine Supported Bounding Locomotion with a Small Compliant Quadruped Robot", in *Proceedings of the 2013 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2013, pp. 3329–3334. DOI: 10.1109/ICRA.2013.6631041.
- [Dup-2017] J. Duperret, B. Kramer, and D. E. Koditschek, "Core Actuation Promotes Self-Manipulability on a Direct-Drive Quadrupedal Robot", in *Proceedings of the 2016 International Symposium on Experimental Robotics (ISER)*, Springer, 2017, pp. 147–159.
- [Pus-2013] J. L. Pusey, J. M. Duperret, G. C. Haynes, R. Knopf, and D. E. Koditschek, "Free-Standing Leaping Experiments with a Power-Autonomous, Elastic-Spined Quadruped", in *Proceedings of SPIE Defense, Security, and Sensing: Unmanned Systems Technology XV*, vol. 8741, SPIE, 2013, 87410W. DOI: 10.1117/12.2016073.
- [Ye-2021] K. Ye and K. Karydis, "Modeling and Trajectory Optimization for Standing Long Jumping of a Quadruped with a Preloaded Elastic Prismatic Spine", in *Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 9208–9214. DOI: 10.1109/IROS51168.2021.9636606.
- [Cap-2023] J. D. Caporale, Z. Feng, S. Rozen-Levy, A. M. Carter, and D. E. Koditschek, "Twisting Spine or Rigid Torso: Exploring Quadrupedal Morphology via Trajectory Optimization", in *Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA)*, London, UK: IEEE, 2023, pp. 1177–1184. DOI: 10.1109/ICRA48891.2023.10160450.
- [Bha-2019] S. Bhattacharya, A. Singla, Abhimanyu, *et al.*, "Learning Active Spine Behaviors for Dynamic and Efficient Locomotion in Quadruped Robots", in *Proceedings of the 2019 28th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, IEEE, 2019, pp. 1–8. DOI: 10.1109/RO-MAN46459.2019.8956332.

- [Li-2023] W. Li, Z. Zhou, and H. Cheng, “Dynamic Locomotion of a Quadruped Robot with Active Spine via Model Predictive Control”, in *Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2023, pp. 1185–1191. DOI: 10.1109/ICRA48891.2023.10160896.
- [Wan-2024] J. Wang, J. Cheng, J. Hu, W. Gao, and S. Zhang, “Spined Torso Renders Advanced Mobility for Quadrupedal Locomotion”, in *Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan: IEEE, 2024, pp. 1457–1463. DOI: 10.1109/ICRA57147.2024.10610113.
- [Ras-2023] *Raspberry Pi 5 Product Brief*, Accessed: 2025-09-01, Raspberry Pi Ltd, 2023.
- [ST-2023a] *STM32H723/733 Microcontrollers Datasheet*, Accessed: 2025-09-01, STMicroelectronics, 2023.
- [ST-2021] *AN5348: Getting Started with STM32H7 Series MCUs*, Application Note, Accessed: 2025-09-01, STMicroelectronics, 2021.
- [Zho-2023] C. Zhou and N. Tsagarakis, “On the Comprehensive Kinematics Analysis of a Humanoid Parallel Ankle Mechanism”, *ASME Journal of Mechanisms and Robotics*, 2023, Available as draft preprint.

7 List of Figures

Figure 1-1	From left to right: MIT Mini Cheetah (MIT), ANYmal (ETH Zurich), Spot (Boston Dynamics).	2
Figure 1-2	Spine flexion and extension in a running cheetah.	2
Figure 1-3	Picture of the quadruped without sideplates	4
Figure 2-1	Rendering of the quadruped with CoG	6
Figure 2-2	General electro-mechanical diagram	7
Figure 2-3	Naming convention	8
Figure 2-4	Overview of communications with usb ports, can buses and can ids . . .	12
Figure 2-5	Standard CAN-FD frame (when not specified bit = 1) from [ST-2021] . .	13
Figure 2-6	FDCAN setup: TX/RX on STM32	14
Figure 2-7	FDCAN setup: Bit Timing	14
Figure 2-8	Data field in equivalent RAM words, [ST-2021]	14
Figure 2-9	Parameters for RAM Offset	15
Figure 2-10	FDCAN Message RAM Offset	15
Figure 2-11	Electronics 3D printed front support	17
Figure 2-12	Rendering of the quadruped	17
Figure 2-13	FBD for torque calculations in yz plane view	18
Figure 2-14	FBD for torque calculations in xz plane view	18
Figure 2-15	Minimum torque results for legs actuators	19
Figure 2-16	Leg design	21
Figure 2-17	FBD for spinal torque calculation in yz plane	22
Figure 2-18	FBD for spinal torque calculation in xz plane	23
Figure 2-19	Minimum torque results for spine actuators	24
Figure 2-20	General view of the spine design.	25
Figure 2-21	Side view of the spine design.	25
Figure 3-1	Origin of the frame on the leg in its initial position for IK and associated dimensions	27
Figure 3-2	Orientation of the coordinate frames of the four legs	27

Figure 3-3	Quadruped with end-effector desired position for front right (FR) leg . . .	30
Figure 3-4	Front right leg view in x-z plane with end effector desire position	31
Figure 3-5	Front-right leg view in leg plane with end effector desire position	31
Figure 3-6	Spine diagram	32
Figure 3-7	Spine dimensions in x-z plane and y-z plane	34
Figure 3-8	Foot trajectory over time (axis have same scale)	37
Figure 3-9	Diagram of the quadruped on an inclined plane	38
Figure 3-10	Rear body balance	38
Figure 3-11	Front body balance	39
Figure 3-12	Foot position on inclined plane	40
Figure 4-1	ROS2 nodes graph	41
Figure 4-2	Function call sequence in <code>leg_command_node.cpp</code>	44
Figure 4-3	Function call sequence for position control in <code>mujoco_node.cpp</code>	45
Figure 4-4	Function call sequence in <code>mujoco_node.cpp</code>	45
Figure 4-5	Function call sequence in <code>hardware_check.cpp</code>	46
Figure 4-6	Rendering of the quadruped with the 19 hinge joints for URDF export . .	52
Figure 5-1	Mass distribution of the quadruped.	59
Figure 5-2	Simulation and results of trot test	61
Figure 5-3	Simulation and results of balance on inclined surface test	61

8 List of Tables

Table 1-1	Specifications of selected quadrupeds compared to the developed prototype.	2
Table 2-1	Moments of inertia at the center of mass of the quadruped ($\text{kg}\cdot\text{m}^2$).	6
Table 2-2	Comparison of electronic architectures for quadruped integration.	11
Table 2-3	Maximum communication frequencies of all devices in the system	12
Table 2-4	Structure of the USB-to-CAN message format	13
Table 2-5	Byte-wise structure of the 8-byte CAN data fields	15
Table 2-6	MIT-mode 8-byte CAN data layout (big-endian, nibble-stuffed)	16
Table 2-7	Motor specifications from manufacturer datasheets	20
Table 2-8	Mass of major leg components	20
Table 2-9	Mass of the major spine components	24
Table 4-1	Joint and leg order in arrays.	41
Table 5-1	Mass distribution of quadruped components.	58

A Anhang

A.1 Code Listings

Collapsed code structure (C++) of leg_command_node.cpp

```
1 class leg_commands {
2 public:
3     explicit leg_commands(const rclcpp::Logger &logger, int &fd_front,
4                           int &fd_rear);
5
6     // Step 1: Message reception
7     std::optional<std::tuple<int,int,float>> handle_cmd(const std::
8               string &txt);
9
10 private:
11     rclcpp::Logger logger_;
12     int &fd_front_, &fd_rear_;
13
14     // Helpers
15     inline uint16_t f2u(float x, float xmin, float xmax, unsigned bits);
16     inline uint8_t to_u8(int v);
17
18     // Step 3: USB frame assembly
19     inline std::array<uint8_t,14> build_usb_frame(const std::array<
20               uint8_t,11> &can_frame);
21     void build_frame(int can_bus, int can_id,
22                     const std::array<uint8_t,8> &payload, std::array<
23               uint8_t,11> &can_frame);
24
25     // Step 2: Payload preparation
26     void prepare_payload_cmd(const std::string &cmd, std::array<uint8_t
27               ,8> &payload);
28     void prepare_payload_mit(std::array<uint8_t,8> &payload,
29                             const std::vector<std::string> &request,
30                             int can_bus, int can_id);
31     void prepare_payload_request(std::array<uint8_t,8> &payload,
32                                 const std::vector<std::string> &request
33                                   , int &can_id);
34
35     // Step 4: USB transmission
36     std::optional<float> send_usb(int leg, const std::array<uint8_t,11>
37               &can_frame, bool wait_reply);
38 };
39
```

Collapsed code structure (C++) of mujoco_node.cpp

Collapsed code structure (C++):

```

1 // --- Step 2: State acquisition (torque mode) ---
2 void get_cur_actuator_pos(mjData* d, const mjModel* m);
3 void get_cur_actuator_vel(mjData* d, const mjModel* m);
4
5 // --- Step 3: Control computation (torque mode) ---
6 void computePIDtorque_fixed(mjData* d, const mjModel* m);
7 void computePDtorque_fixed(mjData* d, const mjModel* m);
8
9 // --- Step 4: Command sending ---
10 void send_cmd_torque();
11 void send_cmd_pos();
12
13 // Utility
14 void incline_plane(const mjModel* m, mjData* d);
15
16 // Main control orchestration
17 void controller(const mjModel* m, mjData* d);
18
19 // ROS2 Integration
20 class mujoco : public rclcpp::Node {
21 public:
22     mujoco();
23     void start_subscription(); // Step 1: Message reception
24 private:
25     // ROS2 publishers/subscribers
26 };

```

Collapsed code structure (C++) of hardware_check.cpp

Collapsed code structure (C++):

```

1 class Publisher : public rclcpp::Node {
2 public:
3     Publisher(); // init + setup_sequence()
4
5 private:
6     // --- Sequencing ---
7     void setup_sequence(); // calls do_startup_sequence(_)
8     void do_startup_sequence(int); // leg==6: all legs; else: one leg
9
10    // --- Debug helpers (optional) ---

```

```

11     void send_position_requests();
12     void send_mit_cst_test();           // MIT kp=5, kd=0.5, pos/vel/tau=0
13     void send_disable_all();           // disable every joint
14
15     // --- Utilities ---
16     static std::string fmt(int leg, int joint, const std::string& tail);
17
18     // --- ROS2 ---
19     rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
20     rclcpp::TimerBase::SharedPtr timer_mit_, timer_; // commented by
        default
21 };

```

Collapsed code structure (C++) of TrotEquations class

Editable parameters and structure (C++):

```

1 class TrotEquations {
2 public:
3     void compute_xyz_goal_pos();       // called every 10 ms
4
5 private:
6     // --- TIME CONSTANTS [s] ---
7     const double T_stride = 2.0;       // stride period
8     const double B        = 0.65;     // duty factor
9     const double T_norm_stance = T_stride * B;
10    const double phi[4] = {0.0, 0.5, 0.5, 0.0}; // FR, FL, RR, RL
11
12    // --- GEOMETRY [m] ---
13    const double Z_base_height      = -0.255;
14    const double X_goal_endeff      = 0.06;
15    const double HEIGHT_apex_swing = 0.02;
16    const double LENGTH_stride     = 0.06;
17
18    // --- STATE (per-leg normalized time over T) ---
19    double t_norm[4] = {0.0};
20
21    // --- Helpers (called inside compute_xyz_goal_pos) ---
22    double smooth_poly(double u);        // u in [0,1]
23    void stance(double t, int leg, double T_norm_stance); // x,y,z on
        ground
24    void swing (double t, int leg, double T_norm_stance); // x,y,z in
        air
25    void x_goal(int leg); void y_goal(int leg, int s, double t_s);
26    void z_goal(int leg, int s, double t_s);

```

```
27 };
```

Collapsed code structure (C++) of IKSolver class

Editable parameters and structure (C++):

```
1 class IKSolver {
2 public:
3     void compute_IK(); // maps foot xyz goals -> [waist, hip, knee] per
        leg
4 private:
5     // Robot geometry [m]
6     const double d1 = 0.06; // waist->hip x-offset
7     const double a2 = 0.14; // femur (hip->knee)
8     const double a3 = 0.141; // tibia (knee->foot)
9 };
10
11 class SpineEquations {
12 public:
13     void compute_spine_actuator_goal(); // currently sets all spine
        goals to 0
14 };
```

Collapsed code structure (C++) of GaitController class

Controller (expanded) and ROS 2 integration (C++):

```
1 class GaitController {
2 public:
3     void update() {
4         trot_eq_.compute_xyz_goal_pos(); // 1) foot trajectories
5         ik_.compute_IK(); // 2) leg joint targets
6         spine_eq_.compute_spine_actuator_goal(); // 3) spine targets (0
            for now)
7     }
8 private:
9     TrotEquations trot_eq_;
10    IKSolver ik_;
11    SpineEquations spine_eq_;
12 };
13
14 class MinimalPublisher : public rclcpp::Node {
15 public:
16     MinimalPublisher(GaitController* controller)
17     : Node("trot_node"), controller_(controller) {
18         // 10 ms: run controller + publish all 15 goals (MuJoCo &
            logging)
```

```

19     publisher_ = create_publisher<std_msgs::msg::Float64MultiArray>(
20         "actuator_goal_rad", 10);
21     timer_ = create_wall_timer(10ms, [this]{
22         controller_>update();
23         publish_actuator_goal_rad_line();    // one-line array log
24     });
25
26     // 30 ms: publish MIT commands with a 10 s kp,kd ramp
27     publisher_mit_ = create_publisher<std_msgs::msg::String>(
28         "leg_command_topic", 10);
29     ramp_start_ = now();
30     timer_mit_ = create_wall_timer(30ms, [this]{ send_MIT_cmd(); });
31 }
32 private:
33     void publish_actuator_goal_rad_line(); // packs 12 leg + 3 spine
        values
34     void send_MIT_cmd();                  // kp,kd ramped over ~10 s
35     GaitController* controller_;
36     rclcpp::TimerBase::SharedPtr timer_, timer_mit_;
37     rclcpp::Publisher<std_msgs::msg::Float64MultiArray>::SharedPtr
        publisher_;
38     rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_mit_;
39 };

```

Erklärung

Ich versichere hiermit, dass ich die von mir eingereichte Abschlussarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.



Garching, 01.09.2025, Federico Cominelli